

Sapientia Erdélyi Magyar Tudományegyetem

Marosvásárhelyi Kar

Számítástechnika szak

Bus4U - Szerver oldal

DIPLOMADOLGOZAT

Témavezető:

S.l.dr.ing. Szántó Zoltán

Hallgató:

Zsigmond Attila

2025

Universitatea „Sapientia” din Cluj-Napoca
Facultatea de Științe Tehnice și Umaniste din Târgu-Mureș
Specializarea Calculatoare

Bus4U - Partea de Server

PROIECT DE DIPLOMĂ

Coordonator științific:

Ș.l.dr.ing. Szántó Zoltán

Student:

Zsigmond Attila

2025

Extras

Scopul aplicației web este de a face sistemele moderne de transport public mai prietenoase cu utilizatorul și mai eficiente, cu un accent deosebit pe planificarea călătoriilor, achiziționarea de bilete și urmărirea autobuzelor. Aplicația permite utilizatorilor să își planifice călătoriile în avans în cadrul rețelei de transport public dintr-un oraș sau o regiune specifică. Utilizatorii își pot achiziționa biletele pentru o rută selectată și pot vizualiza orele de plecare și sosire ale curselor, defalcate pe orașe și stații, asigurând astfel flexibilitate și informații de călătorie precise.

Aplicația oferă, de asemenea, o hartă interactivă care permite urmărirea în timp real a pozițiilor curente ale autobuzelor. Această funcție este deosebit de utilă atunci când apar schimbări neprevăzute în program, deoarece utilizatorii pot fi informați la timp despre întârzieri sau modificări ale traseelor. Pe lângă urmărirea în timp real, sistemul oferă locațiile geografice precise ale stațiilor de autobuz individuale, pe care utilizatorii le pot filtra în funcție de orașe sau rute, facilitând astfel planificarea la care stație să ajungă.

O aplicație separată care rulează pe terminalele POS aflate pe autobuze se ocupă de validarea biletelor și de partajarea continuă a pozițiilor autobuzelor cu serverul central, permițând utilizatorilor să rămână informați despre locațiile în timp real ale autobuzelor.

O altă componentă importantă a aplicației este interfața de administrare utilizată de companiile de autobuze. Această interfață permite gestionarea programelor, adăugarea de noi rute și stații și actualizarea datelor existente. În plus, informațiile despre angajați, inclusiv detaliile șoferilor și ale altui personal, precum și datele tehnice și de operare ale autobuzelor, pot fi gestionate aici. Acest sistem de administrare asigură funcționarea eficientă și întreținerea programelor, permițând companiilor de autobuze să își actualizeze cu ușurință serviciile și să asigure acuratețea.

Tema centrală a lucrării de diplomă este descrierea detaliată a componentelor serverului aplicației web, care asigură gestionarea fluidă a datelor utilizatorilor, a tranzacțiilor de achiziție a biletelor, a informațiilor de program și a urmăririi autobuzelor în timp real.

Cuvinte cheie: transport public, urmărire în timp real, aplicație web, Rails API

Abstract

The purpose of the web application is to make modern public transportation systems more user-friendly and efficient, with a particular focus on trip planning, ticket purchasing, and bus tracking. The application allows users to plan their trips in advance within a specific city or region's public transportation network. Users can purchase tickets for a selected route and view the departure and arrival times of the services, broken down by cities and stops, thus ensuring flexibility and accurate travel information.

The application also offers an interactive map that allows real-time tracking of the current positions of buses. This feature is especially useful when unexpected changes occur in the schedule, as users can be timely informed about delays or route changes. In addition to real-time tracking, the system provides the precise geographical locations of individual bus stops, which users can filter based on cities or routes, making it easier to plan which stop to arrive at.

A separate application running on the POS terminals located on the buses ensures ticket validation and the continuous sharing of the buses' positions with the central server. This feature ensures that all participants in the system have up-to-date information, allowing users to stay informed about the real-time locations of the buses.

Another important component of the application is the administration interface used by the bus companies. This interface allows for the management of schedules, the addition of new routes and stops, and the updating of existing data. Additionally, employee information, including details of drivers and other staff, as well as technical and operational data of the buses, can be managed here. This administration system ensures efficient operations and schedule maintenance, enabling bus companies to easily update their services and ensure accuracy.

The central theme of the thesis is the detailed description of the server-side components of the web application, which ensure the seamless management of user data, ticket purchase transactions, schedule information, and real-time bus tracking.

Keywords: public transportation, real-time tracking, web application, Rails API

Kivonat

A webalkalmazás célja a modern tömegközlekedési rendszerek felhasználóbaráttá és hatékonyabbá tétele, különös tekintettel az utazás megtervezésére, jegyvásárlásra, valamint az autóbuszok követésére. Az alkalmazás lehetővé teszi a felhasználók számára, hogy előre megtervezhessék utazásaikat egy adott város vagy régió tömegközlekedési hálózatán belül. A felhasználók megvásárolhatják jegyeiket egy kiválasztott járatra, valamint megtekinthetik a járatok indulási és érkezési időpontjait, városokra és megállókra bontva, biztosítva ezzel a rugalmasságot és a pontos utazási információkat.

Az alkalmazás egy interaktív térképet is kínál, amelyen valós időben nyomon követhetők az autóbuszok jelenlegi pozíciói. Ez a funkció különösen hasznos, amikor a menetrendben váratlan változások történnek, mivel a felhasználók időben értesülhetnek a késésekről vagy útvonal-módosításokról. A valós idejű követés mellett a rendszer biztosítja az egyes buszmegállók pontos földrajzi helyzetét, amelyeket a felhasználók városok vagy útvonalak alapján szűrhetnek, így könnyebben tervezhetik meg, melyik megállóhoz érkezzenek.

A buszokon található POS terminálokon futó különálló alkalmazás gondoskodik a jegyértékesítésről és az autóbuszok pozíciójának folyamatos megosztásáról a központi szerverrel, így a felhasználók folyamatosan tájékozódhatnak az autóbuszok valós helyzetéről.

Az alkalmazás másik fontos része az adminisztrációs felület, amelyet a busztársaságok használhatnak. Ezen a felületen lehetőség van a menetrendek kezelésére, új útvonalak és megállók felvitelére, valamint a meglévő adatok frissítésére. Továbbá itt kezelhetők az alkalmazottak információi, beleértve a sofőrök és egyéb személyzet adatait, valamint az autóbuszok műszaki és üzemeltetési adatai. Ez az adminisztrációs rendszer biztosítja a hatékony működést és a menetrendek karbantartását, így a busztársaságok könnyedén frissíthetik szolgáltatásaikat és biztosíthatják a pontosságot.

A dolgozat központi témája a webalkalmazás szerver oldali komponenseinek részletes ismertetése, amelyek biztosítják a felhasználói adatok, a jegyvásárlási tranzakciók, a menetrendi információk, valamint a valós idejű autóbusz-követés zökkenőmentes kezelését.

Kulcsszavak: tömegközlekedés, valós idejű követés, webalkalmazás, Rails API

Tartalomjegyzék

1. Bevezető	7
2. Célkitűzések	10
3. Elméleti megalapozás és bibliográfiai tanulmány	12
3.1. A közlekedési rendszerek és a jövőbeli trendek	12
3.2. Hasonló alkalmazások	13
3.2.1. Hatások a közlekedési rendszerre	13
3.2.2. Moovit backend struktúra	14
3.3. Backend architektúra és technológiai háttér	15
3.3.1. REST API és HTTP működése	15
3.3.2. Backend architektúrák	15
3.3.3. Adatbázis-kezelés és skálázhatóság	16
4. Rendszer specifikációi	17
4.1. Felhasználói követelmények	17
4.1.1. Felhasználói weboldal	17
4.1.2. Admin felület	18
4.1.3. Mobil applikáció	18
4.1.4. POS terminál alkalmazás	18
4.2. Rendszerkövetelmények	20
4.2.1. Funkcionális követelmények	20

4.2.2. Nem-funkcionális követelmények	20
5. A rendszer részletes leírása	22
5.1. Technológiai háttér	23
5.1.1. Ruby on Rails	23
5.1.2. PostgreSQL	24
5.1.3. Azure	25
5.2. Az API felépítése	26
5.3. Fontosabb endpointok részletes leírása	26
5.3.1. GET /v1/get_stations_by_city	26
5.3.2. GET /v1/get_departure_times_for_station_in_route	27
5.3.3. POST /generate_a_ticket	29
5.3.4. GET /admin/statistics	31
5.3.5. POST /use_ticket	33
5.3.6. Hitelesítés és jogosultságkezelés	34
5.4. Adatkezelés és adatbázis kapcsolat	35
5.4.1. Adatbázis táblák	35
5.4.2. Objektum-relációs leképzés (ORM)	38
5.5. Projektmenedzsment és verziókövetés	38
5.5.1. Jira	38
5.5.2. Git és GitHub	40
6. Infrastruktúra	43
6.1. Azure infrastruktúra áttekintése	43
6.2. Domain konfiguráció	43
6.3. Adatbázis telepítése és konfigurációja	44
6.4. Szerver telepítése	44
7. Az alkalmazás működése	46
7.1. Felhasználói felület	46

7.1.1.	Főoldal	46
7.1.2.	Jegyek	49
7.1.3.	Menetrend	50
7.1.4.	Megállók és térkép	50
7.2.	Adminisztrációs felület	51
7.2.1.	Kezdőlap	51
7.2.2.	Megállók	52
7.2.3.	Útvonalak	52
7.2.4.	Menetrend	53
7.2.5.	Buszok és alkalmazottak	53
7.3.	POS terminál alkalmazása	53
7.3.1.	GPS oldal	53
7.3.2.	Jegyszkennelő oldal	54
8.	A rendszer tesztelése	56
8.1.	Tesztelési környezet	56
8.2.	Funkcionális tesztelés	56
8.2.1.	API végpontok tesztelése	57
8.2.2.	Unit tesztek	59
8.3.	Biztonsági tesztelés	60
8.3.1.	SQL Injection	60
8.3.2.	XSS (Cross-Site Scripting) Tesztelés	61
8.3.3.	Felfedezett sebezhetőségek	61
9.	Összefoglalás	64
	Irodalomjegyzék	65
A	Függelék	68

Ábrák jegyzéke

4.1. Use-case diagram a felhasználók és az adminisztrátorok lehetőségeiről	19
5.1. A rendszer architektúra diagramja	23
5.2. Az rendszer adatbázisdiagramja	37
5.3. A Jira kanban board áttekintése	40
5.4. A GitHub repository áttekintése	42
6.1. Telepítési diagram	45
7.1. Aktivitás diagram az utazás tervezésének folyamatáról	47
7.2. Megvásárolt jegy	48
7.3. Szekvencia diagram az útvonaltervezés folyamatáról	49
7.4. Szekvencia diagram a jegyvásárlás folyamatáról	50
7.5. Szekvencia diagram a dokumentumok ellenőrzéséről	52
7.6. Busz kiválasztása és GPS követés	54
7.7. Elindított GPS követés	54
7.8. Sikeres jegy érvényesítés	55
7.9. Sikertelen jegy érvényesítés	55
8.1. A GET /v1/get_stations_by_city végpont kérésének paraméterei és válasza . .	57
8.2. A GET /v1/get_available_tickets végpont kérésének paraméterei és válasza . .	58
8.3. A GET /v1/get_bus_locations_by_route végpont kérésének paraméterei és válasza	58
8.4. A GET /v1/tickets_of_user végpont kérésének paraméterei és válasza	59

8.5. A POST /v1/admin/add_station_to_route végpont kérésének paraméterei és válasza	59
8.6. A ZAP által felfedezett sebezhetőségek listája	61

1. fejezet

Bevezető

A modern világban a digitalizáció egyre inkább áthatja mindennapjainkat, a közlekedéstől kezdve a pénzügyi tranzakciókon át egészen az ügyintézésig. Az új technológiák bevezetése nem csupán kényelmesebbé, hanem hatékonyabbá és átláthatóbbá is teheti a különböző szolgáltatásokat. A tömegközlekedés területén azonban még mindig számos kihívással kell szembenéznünk, különösen a kisebb városokban és a regionális utazások során. Bár a nagyvárosokban egyre több helyen elérhető az elektronikus jegyrendszer és a valós idejű járatinformáció, sok vidéki térségben és távolsági közlekedési vonalon ezek a lehetőségek még mindig hiányoznak.

Az autóbuszok használata a közlekedés egyik leginkább környezetbarát formája, hiszen csökkenti a városi forgalmat és a szén-dioxid-kibocsátást. Ennek ellenére a buszközlekedés sokszor nem versenyképes alternatíva a magánautózással szemben, mivel a szolgáltatás minősége és elérhetősége gyakran nem felel meg a modern elvárásoknak. A jelenlegi rendszerek egyik legfőbb problémája a jegyvásárlási lehetőségek korlátozottsága: míg a városi közlekedésben már egyre inkább elérhető a bankkártyás és mobiltelefonos fizetés, a távolsági és regionális buszjáratok esetében sokszor még mindig csak készpénzzel lehet jegyet vásárolni a sofőrnél.

Egy másik fontos hiányosság, hogy sok esetben nem érhető el egy központi, megbízható információforrás a buszok menetrendjéről és aktuális helyzetéről. A buszmegállókban kifüggesztett menetrendek nem mindig naprakészek, és ha egy járat késik vagy kimarad, az utasoknak nincs lehetőségük arra, hogy erről időben értesüljenek. Egy digitális platform lehetőséget biztosíthatna a menetrendek

valós idejű frissítésére, a buszok helyzetének követésére, valamint az utasok tájékoztatására késések esetén. Emellett sok felhasználó számára fontos lenne egy olyan alkalmazás, amely képes előre tervezni az utazásokat, például jegyfoglalási lehetőséggel vagy a járatok közötti átszállási lehetőségek figyelembevételével.

A jegyvásárlás és az utastájékoztatás digitalizálása nemcsak az utazók, hanem a busztársaságok számára is számos előnyt jelenthet. Az online jegyeladási rendszer segítségével pontosabb előrejelzések készíthetők az utasforgalomról, ami lehetővé teszi a járatok hatékonyabb üzemeltetését. Emellett a flottakezelési és adminisztrációs folyamatok is jelentősen egyszerűsíthetők egy digitális rendszer bevezetésével. Az autóbuszok műszaki állapotának és hivatalos dokumentumainak nyilvántartása papíralapon időigényes és kevésbé hatékony, míg egy online rendszer automatizált értesítésekkel segítheti a karbantartási és engedélyezési folyamatokat.

Fontos figyelembe venni azt is, hogy az utasok egyre nagyobb mértékben elvárják a digitális szolgáltatásokat, hiszen az okostelefonok és az internet folyamatos elérhetősége alapvetővé vált a mindennapi életben. Egy felhasználóbarát alkalmazás, amely egy helyen biztosítja az összes szükséges információt és lehetőséget a jegyvásárlásra, nemcsak kényelmesebbé teszi az utazást, hanem növeli is a tömegközlekedés iránti bizalmat. Az elmúlt években a digitális fizetési megoldások és az online utazástervezési eszközök iránti kereslet ugrásszerűen megnőtt, így a tömegközlekedési szolgáltatóknak is alkalmazkodniuk kell a változó utazási szokásokhoz.

A közlekedési rendszerek fejlesztése nem csupán technológiai kérdés, hanem társadalmi és környezetvédelmi szempontból is kiemelkedő jelentőséggel bír. A fenntartható közlekedési formák támogatása elengedhetetlen a nagyvárosi forgalmi dugók és a környezetszennyezés csökkentése érdekében. A buszközlekedés hatékonyabbá tétele és népszerűsítése hozzájárulhat a magánjármű-használat visszaszorításához, ezáltal mérsékelve a károsanyag-kibocsátást. Ha az emberek számára elérhetőbbé és megbízhatóbbá válik a tömegközlekedés, akkor nagyobb eséllyel választják azt a mindennapi utazásaik során, ami hosszú távon egy fenntarthatóbb városi mobilitást eredményezhet.

A rendszer két nagy részre osztható fel. Az egyik a frontend, amely a felhasználók számára látható felületeket, így a weboldalakat és a mobilalkalmazást tartalmazza, míg a másik rész a backend, amely az üzleti logikát, adatkezelést és API-végpontokat biztosítja. A backend részletes ismertetése az én dolgozatomban található meg, míg a frontend működését és implementációját külön dolgozat mutatja

be [1].

Ebben a dolgozatban részletesen ismertetem a backend architektúrát, az API követelményeket, a technológiai választásokat, valamint az adatok tárolásának és kezelésének módját. Kitérünk a fejlesztés során alkalmazott tervezési döntésekre, a megvalósított funkciókra, továbbá a tesztelési módszerekre és a jövőbeni továbbfejlesztési lehetőségekre is. A dolgozat végén összegzem az eredményeket és levonom a projekt kapcsán szerzett tapasztalatokat.

2. fejezet

Célkitűzések

A projekt célja egy modern, hatékony és könnyen használható rendszer létrehozása, amely egyszerűsíti a tömegközlekedés használatát, és népszerűsíti a buszközlekedést a felhasználók körében. Az API biztosítja a különböző funkciók zavartalan működését, míg a kapcsolódó felületek kényelmes hozzáférést nyújtanak az utasok és a busztársaságok számára. A következő főbb célkitűzések mentén épül fel a rendszer:

- **Digitális Jegyvásárlás:** Lehetővé kell tenni, hogy az utasok előre megvásárolhassák jegyeiket, így elkerülve a készpénzes fizetések nehézségeit. Az online tranzakciók nyomon követhetősége nemcsak az utasok, hanem az üzemeltetők számára is átláthatóbbá teszi a folyamatokat.
- **Valós Idejű Buszkövetés:** Egy olyan rendszer kialakítása szükséges, amely segíti az utasokat abban, hogy pontos információt kapjanak a buszok aktuális helyzetéről és várható érkezési idejéről. Az adatok forrása GPS-alapú nyomkövetés vagy külső szolgáltatások lehetnek, biztosítva a megbízható tájékoztatást.
- **Menetrendek és Útvonalak Elérhetősége:** Fontos, hogy a járatok menetrendjei mindig naprakészen és könnyen hozzáférhető módon álljanak rendelkezésre. A megállóban elhelyezett információkon túl online is biztosítani kell a menetrendi adatok lekérdezését és a különböző járatok közötti átszállási lehetőségek áttekintését.
- **Interaktív Térkép és Adatszolgáltatás:** Az API-nak biztosítani kell a térképes felületek által

megjelenített adatok (megállók, járatok, útvonalak) strukturált és hatékony lekérdezhetőségét. Az adatok JSON formátumban, RESTful elvek szerint érhetőek el, támogatva a frontend gyors és dinamikus megjelenítését.

- **Adminisztrációs és Jogosultságkezelő Felület:** Az API támogatja az adminisztrációs funkciók (például járatok, útvonalak, menetrendek, járműpark) adatkezelését, a felhasználók jogosultságainak kezelését és a hozzáférési szintek kontrollját. Biztosítja az adatok validációját, az autentikációt, valamint a naplózási funkciókat a későbbi auditálhatóság érdekében.
- **Felhasználói Élmény és Rendszerhatékonyság:** Az egész rendszer kialakításánál kiemelt szempont a gyors és stabil működés, valamint az átlátható és egyszerű kezelhetőség. A végpontok megfelelő szervezése biztosítja, hogy az utasok számára készült alkalmazások gördülékenyen és késedelem nélkül működjenek.
- **Biztonság és Adatvédelem:** A backend rendszer fontos célja, hogy magas szinten biztosítsa az adatbiztonságot és adatvédelmet. Az API megfelelő hitelesítési és jogosultságellenőrzési mechanizmusokat alkalmaz, valamint támogatja a titkosított kommunikációt az érzékeny adatok védelme érdekében.

Ezek a célkitűzések biztosítják, hogy a rendszer egyaránt megfeleljen az utasok és az üzemeltetők igényeinek. A háttérben működő megoldás felelős a jegyvásárlás, a buszok helyzetének követése és a menetrendi adatok biztosításáért, míg a felhasználók számára fejlesztett felületek a könnyű elérhetőséget és intuitív használatot garantálják. Az átlátható struktúra és a megbízható adatkezelés együttesen járul hozzá a modern, digitális tömegközlekedési rendszer kialakításához.

3. fejezet

Elméleti megalapozás és bibliográfiai tanulmány

A közlekedési rendszerek digitalizációja napjaink egyik kiemelten fontos fejlesztési területe, amely a modern technológia alkalmazásával növeli a közösségi közlekedés hatékonyságát, elérhetőségét és fenntarthatóságát. Az intelligens közlekedési rendszerek, az automatizált jegyvásárlás, valamint a valós idejű forgalmi adatok használata mind hozzájárulnak a hatékonyabb és környezetbarátabb közlekedési megoldások kialakításához.

3.1. A közlekedési rendszerek és a jövőbeli trendek

A közlekedési rendszerek fejlődési iránya jelentős hatással van a Bus4U-hoz hasonló alkalmazások sikerére. A jövőbeni trendek az automatizációra, a megosztott mobilitási formákra és az okos városokkal való integrációra helyezik a hangsúlyt [2]. Ezek a trendek lehetővé teszik az olyan megoldások terjedését, amelyek csökkentik a közlekedési rendszerek környezeti terhelését és javítják az utazási élményt. Az automatizált jegyvásárlás és a valós idejű utazási információk széleskörű alkalmazása tovább növeli a tömegközlekedési eszközök népszerűségét.

A hatékony közlekedési rendszerek tervezése és üzemeltetése érdekében elengedhetetlen a szolgáltatások integrált és felhasználóközpontú kialakítása. Az utazási élményt nagyban javítja a digitális

jegykezelés, az átlátható és pontos menetrendi adatok, valamint a könnyen hozzáférhető információk megléte [3].

3.2. Hasonló alkalmazások

A közlekedési alkalmazások, mint például a Budapest Go, jelentős hatással vannak a modern városi közlekedésre. A Budapest Go a magyarországi főváros tömegközlekedési rendszerének digitális megoldása, amely lehetővé teszi a felhasználók számára, hogy egyszerűen és gyorsan intézzék utazásaik szervezését. Az alkalmazás számos funkcióval rendelkezik, amelyek javítják a felhasználói élményt.

A Budapest Go célja, hogy a város tömegközlekedését még hozzáférhetőbbé és felhasználóbarátabbá tegye. Az alkalmazás főbb jellemzői közé tartozik:

- Valós idejű menetrendek: Az alkalmazás folyamatosan frissíti a buszok, villamosok és metrók érkezési idejét, lehetővé téve a felhasználók számára, hogy pontosan tudják, mikor indul a következő járat.
- Online jegyvásárlás: A felhasználók az alkalmazáson belül jegyeket vásárolhatnak, ami gyorsítja a folyamatot és csökkenti a sorban állás szükségességét.
- Útvonaltervezés: Az alkalmazás lehetővé teszi a felhasználók számára, hogy megtervezzék utazásaikat, figyelembe véve a különböző közlekedési eszközöket és a lehető leghatékonyabb útvonalakat.
- Használati statisztikák: A Budapest Go elemzi a felhasználók utazási szokásait, lehetővé téve a szolgáltatók számára, hogy jobban megértsék a közlekedési igényeket és fejleszthessék szolgáltatásaikat.

3.2.1. Hatások a közlekedési rendszerre

A Budapest Go alkalmazás bevezetése jelentős hatással volt a főváros közlekedési rendszerére. A digitális jegyvásárlás elterjedése csökkentette a készpénzes tranzakciók számát, így gyorsabbá téve a

közlekedési folyamatokat. Ezenkívül a valós idejű információk elérhetősége segít a felhasználóknak a jobb döntéshozatalban, ami hozzájárul a város közlekedési hatékonyságának növeléséhez.

A Budapest Go egy példa arra, hogyan lehet a digitális technológiát alkalmazni a városi közlekedés fejlesztésére. Az alkalmazás által kínált funkciók és a felhasználói élmény folyamatos javítása hozzájárul a közlekedési rendszerek modernizálásához és a fenntartható közlekedés népszerűsítéséhez.

3.2.2. Moovit backend struktúra

A Moovit, mint globális mobilitási szolgáltatás, backend rendszere modern felhőalapú technológiákra épül. Backend architektúrájának alapját Amazon Web Services (AWS) szolgáltatások adják:

- **Adattárolás:** Az adatok tárolására relációs (például PostgreSQL) és NoSQL adatbázisokat (pl. Amazon S3, DynamoDB) használnak. Ez biztosítja az adatok gyors elérését, skálázhatóságát és a szolgáltatások megbízhatóságát.
- **Alkalmazásszerverek:** Az alkalmazásszerverek futtatására Amazon EC2 példányokat használnak, amelyek rugalmas skálázási lehetőséget biztosítanak a felhasználói igények függvényében.
- **AI-alapú szolgáltatások:** A Moovit a szolgáltatásfigyelmeztetések automatikus osztályozásához Apache Airflow-val integrált Amazon SageMaker szolgáltatást használ. A SageMaker segítségével BERT-alapú NLP modelleket képeznek ki, amelyek lehetővé teszik a szöveges információk pontos automatikus feldolgozását, ezáltal növelve a szolgáltatások pontosságát és sebességét.
- **Infrastruktúra és skálázhatóság:** Az Amazon Elastic Compute Cloud (EC2) és az Amazon API Gateway technológiák révén a backend környezet képes nagy terhelés kezelésére, valamint alacsony késleltetésű válaszidők biztosítására globális felhasználói bázis számára is.
- **Folyamatok automatizálása:** A Moovit backend folyamatait (adatgenerálás, modelltréning, ellenőrzés, deployment) Apache Airflow workflow-kezelő eszközzel automatizálták. Ezáltal a rendszer skálázhatóbb, könnyebben karbantartható és gyorsabban adaptálható új igényekhez.

A Moovit backendjének fejlett struktúrája példaértékű a tömegközlekedési alkalmazások számára, biztosítva a felhasználók számára a stabil, gyors és precíz működést még nagy felhasználószám esetén is.

3.3. Backend architektúra és technológiai háttér

A modern webalkalmazások működésének alapját az API-k és az azok által biztosított szolgáltatások jelentik.

3.3.1. REST API és HTTP működése

A REST (Representational State Transfer) architektúra az egyik legelterjedtebb webszolgáltatási megközelítés, amely az erőforrás-alapú kommunikációt helyezi előtérbe [4]. Az API-k HTTP protokollt használnak, amely négy alapvető metódust biztosít:

- **GET** - Erőforrás lekérése.
- **POST** - Új erőforrás létrehozása.
- **PUT** - Meglévő erőforrás módosítása.
- **DELETE** - Erőforrás törlése.

A REST API-k állapotmentesek, így minden egyes kérés önálló, nem igényel szerveroldali állapotmegőrzést. Az adatok általában JSON formátumban kerülnek továbbításra, biztosítva a platformfüggetlenséget és az egyszerű feldolgozást.

3.3.2. Backend architektúrák

A backend rendszerek fejlesztése során két fő architekturális megközelítést különböztetünk meg:

Monolitikus architektúra: Egyetlen egységként működik, amely minden üzleti logikát és adatkezelést magában foglal. Egyszerűbb fejlesztési és telepítési folyamatokkal rendelkezik, de nehezen skálázható [5].

Mikroszolgáltatás-architektúra: Az alkalmazás kisebb, egymástól független szolgáltatásokból áll, amelyek különálló komponensekként működnek. Ez lehetővé teszi a jobb skálázhatóságot és karbantarthatóságot, de nagyobb fejlesztési és üzemeltetési komplexitással jár.

3.3.3. Adatbázis-kezelés és skálázhatóság

A backend rendszer egyik kulcsfontosságú eleme az adatbázis-kezelés, amely biztosítja az adatok hatékony tárolását, lekérdezhetőségét és integritását. A megfelelő adatbázis kiválasztásánál két fő típust különböztetünk meg:

- **Relációs adatbázisok:** Strukturált adatok tárolására használatosak, biztosítják az adatintegritást és a konzisztens tranzakciókezelést. A Bus4U projektben például PostgreSQL került alkalmazásra, amely magas fokú megbízhatóságot, tranzakcióbiztonságot, valamint könnyű karbantarthatóságot biztosít nagyobb adatmennyiségek kezelése esetén is.
- **NoSQL adatbázisok:** Dinamikusan változó vagy kevésbé strukturált adatok tárolására optimálisak. Rugalmasságot biztosítanak az adatok tárolási módjában, továbbá kiválóan skálázhatók horizontálisan. Gyakran alkalmazott megoldások közé tartozik például az Amazon DynamoDB, amelyet nagy adatmennyiség és gyors elérhetőség esetén használnak.

A backend rendszerek skálázhatósága különösen fontos kritérium, amely biztosítja a rendszer hatékony működését változó felhasználószám mellett is. A skálázhatóság két fő módszerrel valósítható meg:

- **Vertikális skálázás:** A rendszer teljesítményének növelése nagyobb erőforrásokkal (például több memória, erősebb CPU biztosítása). Ez egyszerűbb, de hosszabb távon korlátozott megoldás, mivel a fizikai korlátok határt szabhatnak az erőforrásbővítésnek.
- **Horizontális skálázás:** Több, párhuzamosan működő szerver használatával valósul meg. Az így kialakított rendszer terheléelosztás révén képes kezelni a megnövekedett adatforgalmat, rugalmasan alkalmazkodva a változó igényekhez. Ez a megoldás nagyobb rendelkezésre állást, jobb hibakezelést, valamint egyszerűbb kapacitásnövelési lehetőségeket kínál, ugyanakkor komplexebb rendszerfelügyeletet igényel.

4. fejezet

Rendszer specifikációi

4.1. Felhasználói követelmények

4.1.1. Felhasználói weboldal

A felhasználói weboldal bejelentkezés nélkül használható. A főoldalon lehetőség nyílik járatokra keresni dátum, idő és cél alapján. A menetrendeknél megtekinthetők a buszok és megállók órarendjei. A megállók oldalán az összes megálló térképen jelenik meg, és szűrést is lehet alkalmazni. Az élőben frissülő térképen a buszok pozícióját percről percre lehet követni. A jegyek oldalán listázva vannak az eddig megvásárolt jegyek QR-kódjai.

Fiókot létrehozni kizárólag a jegyvásárláshoz szükséges. A főoldalon, ha a felhasználó megtalálta a megfelelő járatot, ráléphet a vásárlás gombra. A fiók regisztrációjához meg kell adni:

- teljes név
- email cím
- új jelszó
- ellenőrző kód (amit a rendszer elküld a megadott email címre)

4.1.2. Admin felület

Az admin felület kizárólag bejelentkezéssel elérhető. Új fiókokat csak az adott cég vezetője tud létrehozni az "Alkalmazottak" oldalon. Az adminoknak lehetőségük van:

- a megállók listájának szerkesztésére
- az útvonalak összes tulajdonságának módosítására (meglátogatott megállók, indulási időpontok, jegyárak)
- a főoldalon elérhető statisztikák megtekintésére (eladott jegyek száma, megtett kilométer, bevétel)
- a buszok adatainak kezelésére (útdó, biztosítás, műszaki vizsga, buszok kivonása forgalomból, új buszok hozzáadása)

4.1.3. Mobil applikáció

A Flutter alapú mobil applikáció ugyanúgy működik, mint a felhasználóknak szánt weboldal, így ugyanazok a követelmények vonatkoznak erre is.

4.1.4. POS terminál alkalmazás

A POS terminál alkalmazás használatához szükség van egy sofőr szerepkörű admin fiókra. Bejelentkezés után lehetőség van egy adott busz követésének elindítására, valamint a jegyszkennelés működtetésére.

A rendszer használati eset diagramja az 4.1-es ábrán látható.

4.2. Rendszerkövetelmények

4.2.1. Funkcionális követelmények

A felhasználói felület minden oldalán van egy kis form, amelyet kitöltve le lehet kérni az adatbázisból az adott oldalon megjelenítendő adatokat. Ilyenre példa van a főoldalon is, ahol meg kell adni az indulás helyét, a cél helyét (települést) és opcionálisan az ezekhez tartozó megállót, egy-egy lenyíló listából kiválasztva az adatokat. A települések betöltődnek az oldallal együtt, a megállók listája viszont csak a város kiválasztása után, mindig csak az adott településen találhatóak jelennek meg. Ezen kívül a főoldalon van egy dátumválasztó és egy időválasztó is, ezek alapértelmezetten az aktuális dátummal és idővel vannak kitöltve. Ha a form ki van töltve, a keresés gomb lenyomására az oldal elküldi a bevitt adatokat a backend szerverre, amely válaszként elküld egy listát a megfelelő utazási lehetőségekkel. Ezek az adatok JSON formátumban, HTTP protokollon keresztül közlekednek a frontend és backend között. A többi oldal is hasonlóan működik, először az adatokat kell megadni, aztán a gomb lenyomására pillanatok alatt megjelenik a képernyőn a szerver válasza a bevitt adatokra.

A különböző formokon kívül, a weboldalon található térképek is API hívásokkal működnek. A rendszer először betölti az alap térképet a MapBox API segítségével, aztán amint a felhasználó kiválasztotta a megtekinteni kívánt útvonalat a Menetrend vagy az Élő térkép lapon, a weboldal a saját adatbázisunkból kéri le a hozzá tartozó megállók koordinátáit, ezeket megjeleníti a térképen, végül ismét a MapBox API-t használva útvonal-tervet generáltat. Ez úgy működik, hogy a rendszer elküldi a megállók listáját és visszakap egy sok pontból álló új listát a MapBox-tól, ezeket összekötve sok kis vonallal, kirajzolódik a részletes útvonal a térképen.

4.2.2. Nem-funkcionális követelmények

Szervezéssel kapcsolatos követelmények

- **Programozási nyelvek:** Ruby, Dart, HTML, CSS, JavaScript
- **Keretrendszerek:** Ruby on Rails, Flutter, Vue.js
- **IDE:** Visual Studio Code

- **Verziókövetés:** Git és GitHub
- **Adatbázis:** PostgreSQL
- **Cloud szolgáltatás:** Azure
- **Webhoszt:** Netlify
- **Csoportos kommunikáció:** Slack

Termékkel kapcsolatos követelmények

A felhasználói weboldal és az admin felület használatához modern, internetezésre alkalmas eszközre van szükség, legalább 2018-as vagy frissebb böngészővel:

- **Chrome:** 87 vagy újabb
- **Firefox:** 78 vagy újabb
- **Safari:** 14 vagy újabb
- **Edge:** 88 vagy újabb

Az applikáció használatához internettel és GPS antennával rendelkező okostelefon vagy tablet szükséges, a következő minimális követelményekkel:

- **Operációs rendszer:** Android 5.0 vagy iOS 8.0 és újabb
- **Szabad tárhely:** minimum 120 MB
- **Memória:** 516–1024 MB

A POS terminál alkalmazás futtatásához egy Sunmi V2 PRO POS terminál ajánlott. Alternatívaként, ha nem áll rendelkezésre, bármely kamerával és GPS antennával ellátott, minimum Android 5.0 vagy iOS 8-at futtató eszköz is használható:

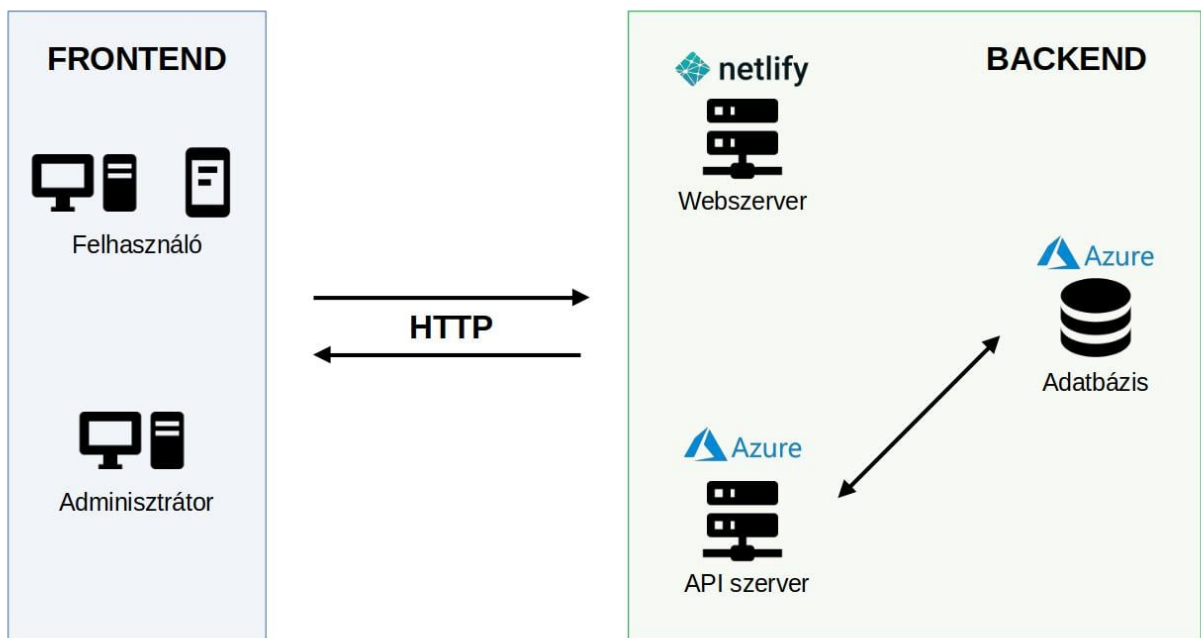
- **Szabad tárhely:** legalább 100 MB
- **Memória:** minimum 510 MB

5. fejezet

A rendszer részletes leírása

Rendszerünk egy Ruby on Rails keretrendszerben írt API-ból, egy Vue.js alapú webes felhasználói- és admin felületről, egy Flutter alkalmazásból a felhasználók számára, valamint egy Flutter keretrendszerben készült, de végül natív Androidként implementált POS terminál alkalmazásból áll. Az API a REST architektúrát követi, lehetővé téve a különböző erőforrások, például buszok, járatok és jegyek kezelését. A felhasználói és admin weboldal Vue-ban készült, a Vuetify elemek felhasználásával, ami lehetővé tette az egyszerű és hatékony felhasználói felületek gyors fejlesztését. A Vue.js keretrendszer előnye, hogy komponensekből áll, így átlátható és jól rendszerezett webalkalmazás felépítést tesz lehetővé. Ezen kívül a Vuetify komponens-készlete megkönnyítette az UI elemek létrehozását, gyorsítva ezzel a fejlesztési folyamatot. Az alkalmazás Flutterben készült, Dart programozási nyelvet használva, amely lehetővé teszi, hogy ugyanazt a kódbázist használjuk iOS és Android platformokra is, így jelentősen csökkentve a fejlesztési időt és költségeket. A POS terminál alkalmazása eredetileg Flutterben készült, de végül natív Android alkalmazásként lett implementálva a nagyobb kompatibilitás érdekében. Ez biztosítja, hogy a rendszer megfelelően működjön a POS terminálokkal, valamint minden szükséges tanúsítvánnyal kompatibilis legyen. A Vue.js alapú weboldalakat, illetve a felhasználóknak szánt Flutter alkalmazást Portik Szabolcs kollegám mutatja be részletesebben egy másik dolgozat keretein belül.

A rendszer komponenseinek kapcsolatát az 5.1 ábra szemlélteti.



5.1. ábra. A rendszer architektúra diagramja

5.1. Technológiai háttér

A Bus4U alkalmazás digitális megoldásai mögött modern webtechnológiák állnak, amelyek lehetővé teszik a felhasználói élmény folyamatos fejlesztését és a szolgáltatások hatékony működését. Az alkalmazás backendje Ruby on Rails keretrendszert használ, amely az egyik legnépszerűbb választás webes alkalmazások fejlesztésére.

5.1.1. Ruby on Rails

A *Ruby on Rails* egy népszerű és széles körben használt webes keretrendszer, amely a Ruby programozási nyelvre épül. A Rails két alapvető elv alapján működik: a „*Convention over Configuration*” és a „*Don't Repeat Yourself*”. Ezek az elvek azt eredményezik, hogy a fejlesztőknek kevesebb kóddal kell dolgozniuk, mivel a Rails konvenciói automatikusan meghatározzák a konfigurációkat, így csökkentve a redundáns kódot és növelve a fejlesztési folyamat hatékonyságát [6, 7]. Ez különösen hasznos

nagyobb projektek esetén, mivel elősegíti a gyorsabb fejlesztést és az egyszerűbb karbantartást, amely a gyorsan változó üzleti igények mellett nélkülözhetetlen.

A Rails *MVC (Model-View-Controller)* architektúrára épül, amely logikailag elkülöníti az alkalmazás különböző részeit: az adatkezelést (Model), a felhasználói felületet (View), és az üzleti logikát (Controller). Ez a struktúra átláthatóbbá teszi a kódot, mivel a különböző funkciók külön-külön karbantarthatók, anélkül hogy zavarnák egymást, valamint megkönnyíti az új funkciók hozzáadását is. A Rails alkalmazásokhoz számos „gem”, vagyis könyvtár érhető el, amelyek előre elkészített megoldásokat kínálnak, így még a komplexebb funkciók integrálása is könnyebb és biztonságosabb lehet [6].

A Rails további előnye, hogy natív módon támogatja *RESTful API*-k létrehozását. A RESTful megközelítés egyértelmű, szabványosított formát biztosít az API-k számára, amely megkönnyíti az alkalmazások közötti integrációt, például egy mobilalkalmazás és a backend közötti kommunikáció során. Ennek a felépítésnek köszönhetően a Rails alkalmazások egyszerűen összekapcsolhatók más rendszerekkel és frontend megoldásokkal [7].

Végül, a Rails beépített tesztelési eszközei lehetőséget adnak az automatizált tesztelésre, amely hozzájárul a szoftverek magasabb megbízhatóságához és a hibamentes működés biztosításához a fejlesztés minden szakaszában. Az ilyen automatizált tesztelési funkciók szintén csökkentik a karbantartási költségeket, hiszen a kódminőség magasabb szinten tartható, és gyorsan azonosíthatók a hibák [6, 7].

5.1.2. PostgreSQL

A *PostgreSQL* egy *nyílt forráskódú, objektum-relációs adatbázis-kezelő rendszer*, amely széles körben ismert teljesítményéről, rugalmasságáról, és különösen alkalmas komplex adatszerkezetek és nagy adattömegek kezelésére. Támogatja a fejlett adattípusokat, bonyolult lekérdezéseket, és párhuzamos műveleteket, amelyek különösen hasznosak a Bus4U-hoz hasonló, valós idejű adatokat feldolgozó rendszerekben [8, 9].

A PostgreSQL előnyei a Bus4U számára:

- Teljesítmény optimalizálása és bonyolult lekérdezések kezelése: A PostgreSQL kiválóan kezeli

a párhuzamos lekérdezéseket és optimalizált módon képes kezelni nagy mennyiségű adatot és komplex lekérdezési struktúrákat. Ez biztosítja az alkalmazás gyors, megbízható működését, és lehetőséget nyújt az adatbázis hatékony méretezésére és finomhangolására az adatmennyiség növekedésével [9].

- Skálázhatóság: Nagyobb adatmennyiség vagy felhasználói terhelés esetén a PostgreSQL támogatja a függőleges és vízszintes skálázást. Ezen túlmenően replikációval biztosítható a magas rendelkezésre állás, amely kritikus szempont a nagy terhelés alatt működő rendszerekben. Így a PostgreSQL hosszú távú, fenntartható teljesítményt és magas rendelkezésre állást biztosít, amely elengedhetetlen a Bus4U megbízható működéséhez [8].

5.1.3. Azure

Az *Azure* egy olyan nagyvállalati szintű, biztonságos és skálázható felhőplatform, amely számos lehetőséget kínál modern alkalmazások fejlesztéséhez és működtetéséhez. Az Azure infrastruktúrája révén biztosítható a folyamatos és megbízható működés, míg a platform rugalmassága lehetővé teszi az erőforrások hatékony kezelését és a skálázhatóságot [10, 11]. Az Azure használatával elkerülhetők a hagyományos szerver infrastruktúrák fizikai korlátai, és olyan erőforrások válnak elérhetővé, amelyek az alkalmazás igényei szerint rugalmasan méretezhetők.

Az Azure előnyei a Bus4U számára:

- Skálázhatóság és rugalmasság: Az Azure automatikusan alkalmazkodik a változó terheléshez, amely lehetővé teszi, hogy a rendszer gördülékenyen kezelje a megnövekedett felhasználói forgalmat. Ez az adaptív megközelítés egy modern, folyamatosan növekedő alkalmazás számára elengedhetetlen, különösen tömegközlekedési rendszerek esetében, ahol a felhasználói igények időben változhatnak [10].
- Biztonság és adatvédelem: Az Azure erőteljes adatvédelmi szabályokat és fejlett biztonsági protollokat alkalmaz. Ezek közé tartozik a többfaktoros hitelesítés, a hálózati tűzfalak, valamint az adattitkosítás, amelyek biztosítják, hogy a Bus4U felhasználóinak adatai biztonságban legyenek. A platform megfelel a szigorú adatvédelmi szabványoknak, ami kulcsfontosságú egy olyan alkalmazás számára, amely szenzitív felhasználói adatokat kezel [11].

5.2. Az API felépítése

A Bus4U API a REST architektúrát követi, és számos végpontot biztosít a különböző erőforrások kezeléséhez, mint például:

- Buszok kezelése: buszok létrehozása, frissítése, törlése.
- menedzselése: járatok lekérése és ütemezése.
- Jegyek és bérletek: jegyek és bérletek vásárlása és kezelése.

5.3. Fontosabb endpointok részletes leírása

5.3.1. GET /v1/get_stations_by_city

Ez az endpoint lehetővé teszi a megállók listájának lekérését egy adott város alapján.

HTTP metódus: GET

Kérés paraméterei:

- **city_uid** (*string, kötelező*): A város egyedi azonosítója, amely alapján a megállók lekérdezése történik.

Példa kérés:

GET https://api.bus4u.online/v1/get_stations_by_city?city_uid=CTY_N11XH

Sikeres válasz (HTTP 200):

```
{
  "stations": [
    {
      "station_uid": "STT_EGAVX",
      "name": "Sapientia",
      "address": "Str. Calea Sighisoarei nr. 2",
```

```

        "longitude": "24.59883",
        "latitude": "46.52339"
    },
    {
        "station_uid": "STT_6C03Q",
        "name": "Dedeman",
        "address": "Bul. 1 Decembrie nr. 189",
        "longitude": "24.59941",
        "latitude": "46.52678"
    }
]
}

```

Hibás válasz (például ismeretlen city_uid esetén) – HTTP 404:

```

{
    "error": "Could not find the city!"
}

```

Lehetséges válaszkódok:

- **200 OK** – A megállók sikeresen lekérdezve.
- **404 Not Found** – A megadott city_uid nem létezik, vagy nincs ilyen város az adatbázisban.

5.3.2. GET /v1/get_departure_times_for_station_in_route

Ez az endpoint lekérdezi az indulási időpontokat egy adott megállóra, egy adott útvonal mentén.

HTTP metódus: GET

Kérés paraméterei:

- **route_uid** (*string, kötelező*): Az útvonal egyedi azonosítója.
- **station_uid** (*string, kötelező*): A megálló egyedi azonosítója.

Példa kérés:

GET [https://api.bus4u.online/v1/get_departure_times_for_station_in_route?
route_uid=RTE_123ABC&station_uid=STT_XYZ456](https://api.bus4u.online/v1/get_departure_times_for_station_in_route?route_uid=RTE_123ABC&station_uid=STT_XYZ456)

Sikeres válasz (HTTP 200):

```
[
  {
    "name": "Munkanap",
    "fare": 6.0,
    "departure_times": ["06:30", "07:15", "08:00"]
  },
  {
    "name": "Szombat",
    "fare": 6.0,
    "departure_times": ["07:00", "09:30", "11:00"]
  }
]
```

Hibás válaszok:

– **HTTP 404** – Ha a megadott útvonal vagy megálló nem található:

```
{
  "error": "Station or route not found!"
}
```

– **HTTP 404** – Ha az adott útvonal nem érinti a megállót:

```
{
  "error": "The bus does not stop at the
```



```
        specified stop on the specified route"
    }
}
```

- **HTTP 404** – Ha nincs menetrend az adott kombinációhoz:

```
{
    "error": "No departure times found for the
    specified station on the specified route"
}
```

Lehetséges válaszkódok:

- **200 OK** – Az indulási időpontok sikeresen lekérdezve.
- **404 Not Found** – A megadott paraméterek alapján nem található adat (útvonal, megálló, kapcsolat, vagy menetrend).

5.3.3. POST /generate_a_ticket

Ez az endpoint lehetővé teszi új jegy vagy jegyek generálását egy felhasználó számára. A jegyek mentésre kerülnek az adatbázisban, és a vásárló értesítést kap emailben. A generált jegyek alapértelmezetten érvényesek, de még nem fizetettek.

HTTP metódus: POST

Hitelesítés szükséges: Igen (bejelentkezett felhasználó)

Kérés törzse (application/json):

```
{
    "company_uid": "CMP_123456",
    "route_uid": "RTE_ABC123",
    "from_station_uid": "STT_START",
```

```
"to_station_uid": "STT_END",
"type": "standard",
"ticket_price": 15,
"quantity": 2
}
```

Paraméterek:

- `company_uid` (*string*, kötelező): A társaság azonosítója.
- `route_uid` (*string*, kötelező): Az útvonal azonosítója.
- `from_station_uid` (*string*, kötelező): A felszállási megálló azonosítója.
- `to_station_uid` (*string*, kötelező): A leszállási megálló azonosítója.
- `type` (*string*, kötelező): A jegytípus (pl. `standard`, `student` stb.).
- `ticket_price` (*double*, kötelező): A jegy ára.
- `quantity` (*integer*, kötelező): A generálandó jegyek száma.

Sikeres válasz (HTTP 200):

```
[
  "TCK_1A2B3C",
  "TCK_4D5E6F"
]
```

A válaszban a sikeresen létrehozott jegyek egyedi azonosítói szerepelnek.

Hibás válaszok:

- **HTTP 401 Unauthorized** – Ha a felhasználó nincs bejelentkezve:

```
{
  "error": "The user is not logged in"
}
```

- **HTTP 422 Unprocessable Entity** – Ha legalább egy jegy mentése sikertelen:

```
{  
    "error": "Cannot create one or more tickets"  
}
```

Jegyek létrehozásának logikája:

- A jegyek `is_valid` attribútuma `true` lesz.
- A `ticket_price` értékét **100-zal felszorozza**, hogy baniban legyen tárolva. Ezt a Stripe működése követeli meg.
- A `expiration_date` automatikusan 1 hónap a vásárlástól számítva.
- A felhasználó email-címére értesítés kerül kiküldésre a vásárlásról.

5.3.4. GET /admin/statistics

Ez az endpoint statisztikai adatokat szolgáltat az adott társasághoz tartozó eladott jegyek alapján, különböző időintervallumokra (havi, éves, összesített). Az adatokat az adminisztrációs felület jeleníti meg a főoldalon.

HTTP metódus: GET

Hitelesítés szükséges: Igen (adminisztrátor jogosultság)

Lekérdezési paraméterek:

- `company_uid` (*string*, kötelező): A lekérdezni kívánt társaság egyedi azonosítója.

Példa kérés:

GET https://api.bus4u.online/v1/admin/statistics?company_uid=CMP_123456

Sikeres válasz (HTTP 200):

```
{  
  "month_tickets": 142,  
  "month_income": 217500,  
  "month_users": 89,  
  "year_tickets": 1123,  
  "year_income": 1723000,  
  "year_users": 456,  
  "all_tickets": 5238,  
  "all_income": 7932000,  
  "all_users": 2031  
}
```

Válaszmezők:

- month_tickets: Az adott hónapban eladott jegyek száma.
- month_income: Az adott hónapban befolyt összeg.
- month_users: Az adott hónapban vásárló egyedi felhasználók száma.
- year_tickets: Az idei évben eladott jegyek száma.
- year_income: Az idei év bevétele.
- year_users: Az idei évben vásárló egyedi felhasználók száma.
- all_tickets: Az összes eladott jegy száma a társaság fennállása óta.
- all_income: Az összes bevétel.
- all_users: Az összes vásárló egyedi felhasználók száma.

Hibás válaszok:

- **HTTP 422 Unprocessable Entity** – Ha nem található a társaság, vagy nincs jegy:

```
{
  "error": "Invalid company uid!"
}
```

- **HTTP 422 Unprocessable Entity** – Ha nincs jegy az adott társasághoz:

```
{
  "error": "No tickets found"
}
```

5.3.5. POST /use_ticket

Ez az endpoint lehetővé teszi egy jegy érvényesítését. A jegy csak egyszer használható, és csak a lejárat dátumán belül.

HTTP metódus: POST

Hitelesítés szükséges: Igen

Törzstesti paraméterek (JSON):

- `ticket_uid` (*string*, kötelező): A jegy egyedi azonosítója.

Példa kérés:

POST https://api.bus4u.online/v1/admin/use_ticket

Content-Type: application/json

```
{
  "ticket_uid": "TCK_98AF3"
}
```

Sikeres válasz (HTTP 202):

```
{  
  "success": "The ticket is validated successfully"  
}
```

Hibás válaszok:

- **HTTP 404 Not Found** – Ha nem található jegy az adott UID alapján:

```
{  
  "error": "The ticket UID is invalid"  
}
```

- **HTTP 422 Unprocessable Entity** – Ha a jegy már lejárt vagy már érvénytelenített:

```
{  
  "error": "The ticket is used or expired"  
}
```

Megjegyzés: Sikeres érvényesítés esetén a rendszer beállítja a jegyet felhasználtként (`is_valid = false`), így az később nem használható újra.

5.3.6. Hitelesítés és jogosultságkezelés

Az API biztonságos használata érdekében JWT (JSON Web Token) alapú token-hitelesítést alkalmazunk. A JWT egy nyílt szabványú (RFC 7519) token-formátum, amely biztonságos adatátvitelt biztosít a kliens és a szerver között.

A JWT tokenek segítségével a felhasználók és adminisztrátorok azonosítása gyorsan és biztonságosan történik, anélkül, hogy minden kérsnél újbóli bejelentkezés szükséges lenne. Az alkalmazott Rails könyvtárak a következők:

- **devise**: biztosítja az alapvető hitelesítési logikát, beleértve a regisztrációt, bejelentkezést, jelszókezelést.
- **devise_token_auth**: kiegészíti a devise-t JWT alapú tokenkezelési funkciókkal, amely lehetővé teszi a stateless (állapotmentes) autentikációt API-k számára.

A JWT token alapú autentikáció folyamata:

1. A kliens elküldi a bejelentkezési adatokat (e-mail, jelszó) a szervernek.
2. A szerver a sikeres autentikáció után egy JWT tokenet generál, amely tartalmazza a felhasználóhoz kapcsolódó alapvető információkat, valamint a token érvényességi idejét.
3. A kliens a további kérései során a kapott tokenet a HTTP fejlécben, `Authorization: Bearer <JWT_TOKEN>` formátumban küldi el a szervernek.
4. A szerver ellenőrzi a tokenet, és ha érvényes, feldolgozza a kérést, visszaküldi a választ.

5.4. Adatkezelés és adatbázis kapcsolat

Az API végpontjai a PostgreSQL adatbázissal kommunikálnak. Az adatbázis tranzakciókat az ActiveRecord ORM (Object-Relational Mapping) segítségével kezeljük, amely lehetővé teszi az adatok egyszerű és hatékony lekérését, beszúrását és frissítését.

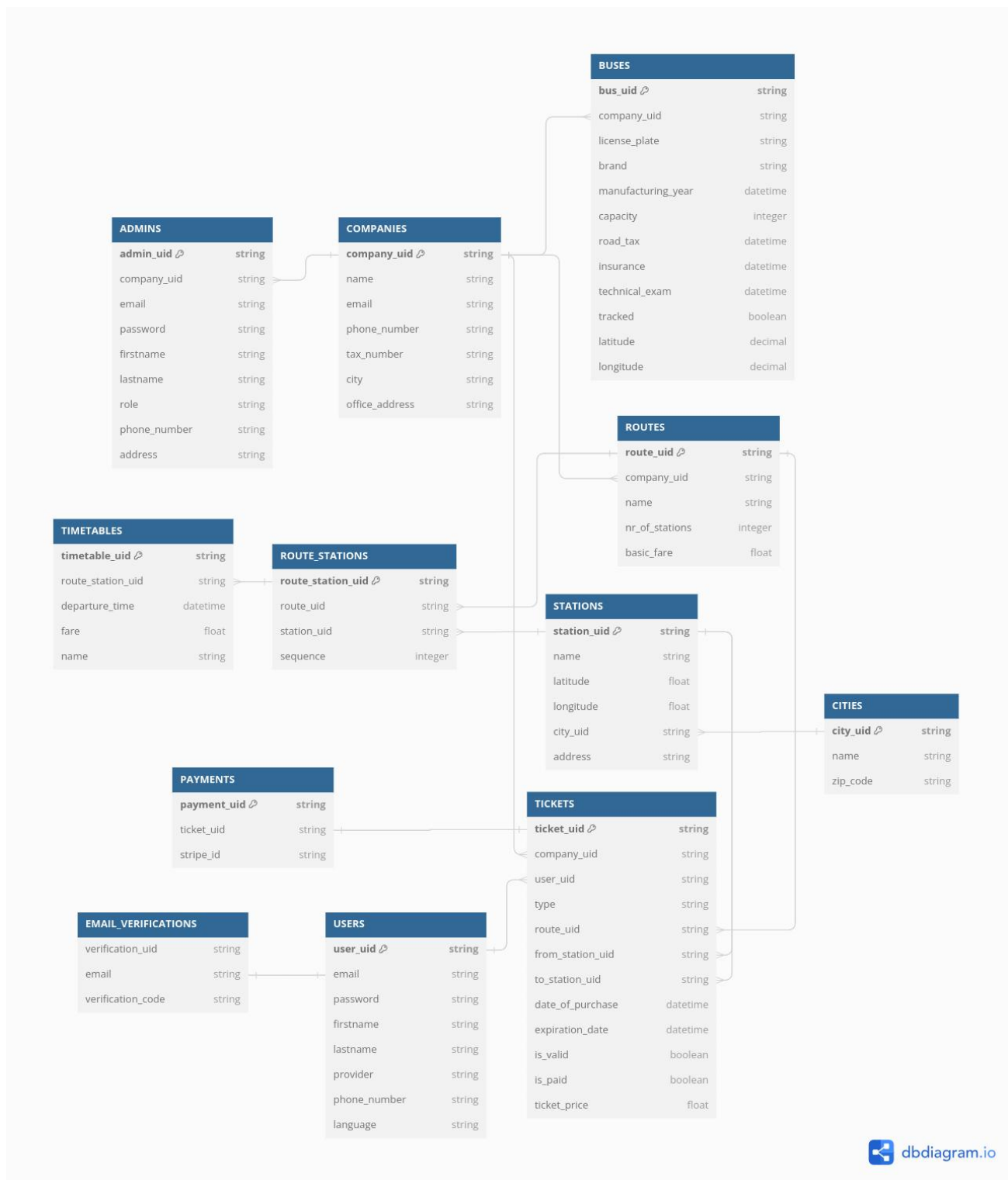
5.4.1. Adatbázis táblák

A PostgreSQL adatbázisunk több, egymással összefüggő táblából áll, amelyek a rendszer különböző entitásait és azok kapcsolatait modellezzik. Ezek a táblák a következők:

- **Companies**: Azon cégek adatai, akikkel szerződést kötött a Bus4U.
- **Admins**: Különböző cégekhez tartozó adminisztrátorok adatai. Szerepük lehet “boss” vagy “driver”.

- **Buses:** A cégekhez tartozó autóbuszok adatai.
- **Routes:** A cégek saját útvonalai.
- **Stations:** A rendszerben található buszmegállók összessége.
- **Route_stations:** Egy cég adott útvonalán keresztülhaladó megállók.
- **Timetables:** Egy útvonal megállójának indulási időpontjai és ára nap szerinti felosztásban.
- **Cities:** Az összes város, amelyen áthaladnak útvonalak.
- **Users:** A Bus4U felhasználói a fontosabb adataikkal.
- **Email_verifications:** Az adott email címekhez tartozó regisztrációkor felhasználandó kódok.
- **Tickets:** A felhasználókhöz tartozó jegyek.
- **Payments:** Stripe fizetések eltárolása.

A rendszer adatbázis-struktúráját és a táblák közötti kapcsolatokat az 5.2 ábra mutatja be részletesen.



5.2. ábra. Az rendszer adatbázisdiagramja

5.4.2. Objektum-relációs leképzés (ORM)

A backend API adatbázis-kezelése a Ruby on Rails keretrendszer beépített ActiveRecord objektum-relációs leképzőjével (ORM) történik. Az ORM lehetővé teszi az alkalmazás számára, hogy közvetlenül Ruby osztályok és objektumok formájában kezelje az adatbázis tábláit, így egyszerűsítve az adatbázis-műveleteket és csökkentve a fejlesztési időt.

Az ActiveRecord fő előnyei közé tartozik:

- Egyszerű adatlekérdezés: A komplex SQL utasítások helyett Ruby nyelven írhatók a lekérdezések.
- Adatkapcsolatok automatikus kezelése: Egyértelművé teszi az adatok közötti kapcsolatokat (például `belongs_to`, `has_many`, `has_one`).
- Validációk és tranzakciók egyszerű kezelése: Adatvalidáció, tranzakciókezelés és hibaüzenetek egyszerű integrációja az üzleti logikába.
- Migrációk támogatása: Könnyű adatbázis-struktúra módosítás a migrációs fájlokon keresztül.

Az ActiveRecord használata jelentősen egyszerűsíti a fejlesztési folyamatot, lehetővé téve, hogy a fejlesztők elsősorban az üzleti logikára összpontosítsanak, miközben a rendszer hatékonyan és biztonságosan kommunikál a PostgreSQL adatbázissal.

5.5. Projektmenedzsment és verziókövetés

A projekt fejlesztése során több eszközt alkalmaztunk a feladatok nyomon követésére, a fejlesztési folyamat szervezésére és a forráskód verziókövetésére.

5.5.1. Jira

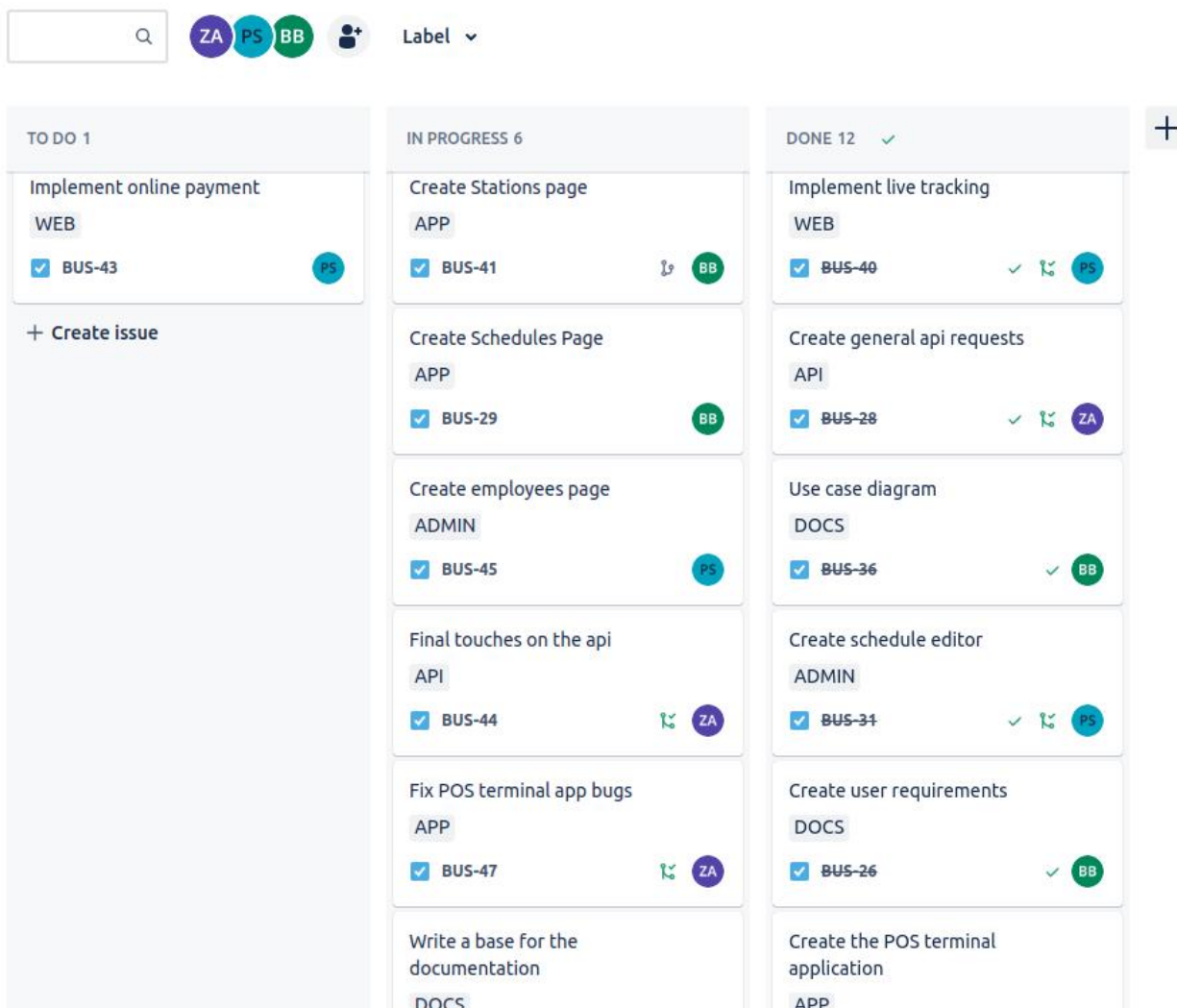
A projektmenedzsment során a Jira kanban boardját használtuk, amely az Atlassian által fejlesztett projekt-követő platform. A Jirát választottuk, mert egy olyan hatékony eszköz, amely egyszerre letisztult, ám funkciókban komplex, így könnyedén igazítható a projekt igényeihez.

A Kanban tábla három fő oszlopból állt, amelyeket az 5.3 ábra szemléltet:

- To do: Ide kerültek azok a feladatok, amelyeket még nem kezdtünk el.
- In progress: Ebben az oszlopban azok a feladatok szerepeltek, amelyek éppen folyamatban voltak.
- Done: Az elvégzett feladatok ebbe az oszlopba kerültek, jelezve azok befejezését.

Minden feladatot hozzárendeltünk egy konkrét tulajdonoshoz, valamint címkével láttunk el, amelyek megjelölték, hogy a munka épp a rendszer melyik komponensére vonatkozik: API, felhasználói felület, admin felület, vagy mobil alkalmazás. A feladatok ilyen címkézése segített abban, hogy pontosan tudjuk, ki mivel foglalkozik, illetve milyen állapotban van a fejlesztés különböző területein végzett munka.

BUS board



5.3. ábra. A Jira kanban board áttekintése

5.5.2. Git és GitHub

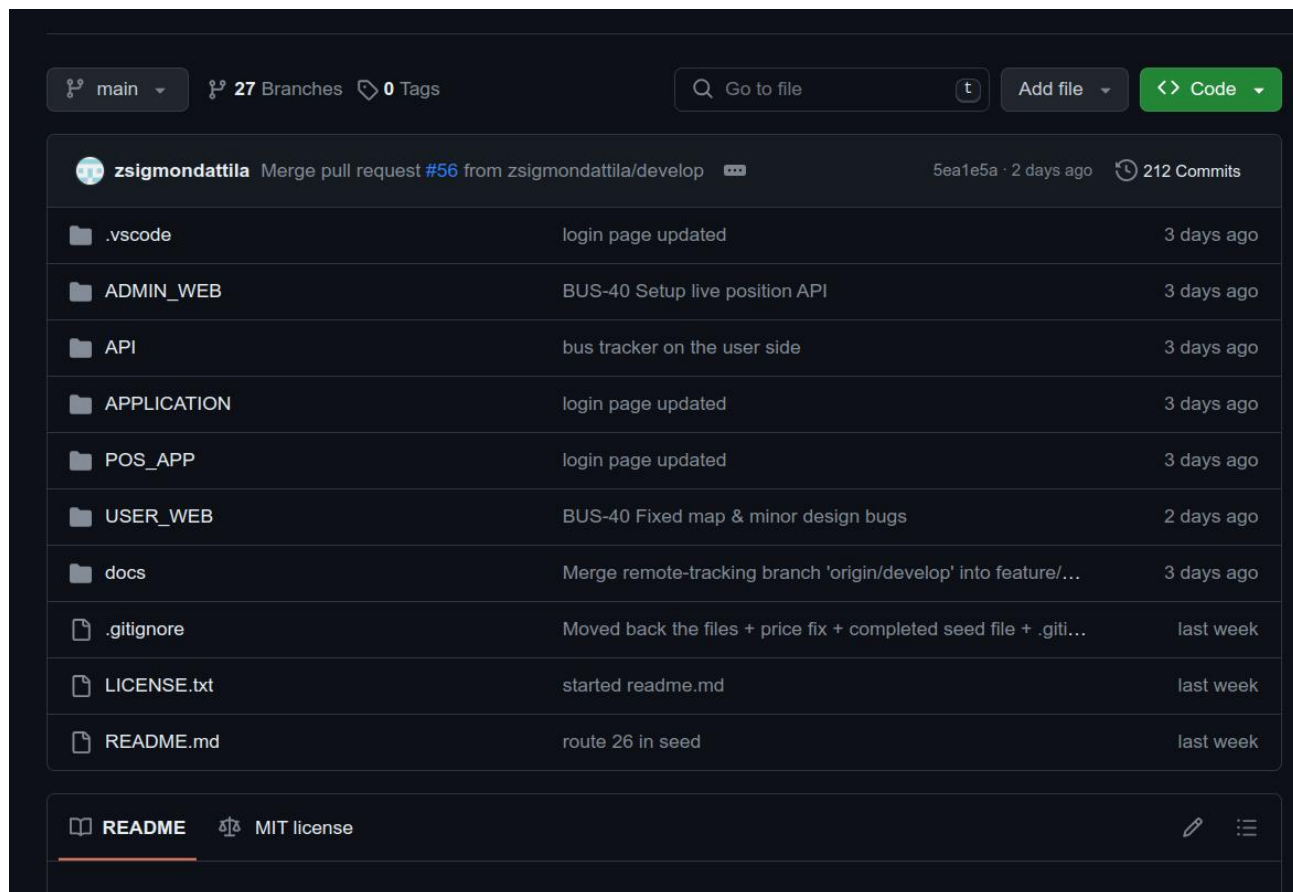
A projekt forráskódjának verziókezelését a Git verziókezelő rendszer segítségével oldottuk meg, míg a kód tárolására a GitHub platformot használtuk. A GitHub lehetőséget biztosított a pull request-ek hatékony kezelésére, amelyekkel minden változtatást egyszerűen tudtunk jóváhagyni. Ez a megközelítés a kód visszakövethetőségét és dokumentálását is jelentősen megkönnyítette, hozzájárulva a kód minőségének javításához és a fejlesztési folyamat átláthatóságához. A 5.4 ábra bemutatja a GitHub

repository áttekintését, ahol látható a main branch mappaszerkezete.

A projekt struktúráját három fő branch köré szerveztük:

- main branch: Ebbe az ágba kizárólag a már teljesen tesztelt és stabil kódrészletek kerültek be, amelyeket időszakosan publikáltunk. Ez az ág szolgált a hivatalos, működőképes verzió tárolására.
- develop branch: A develop branch-et a fejlesztés és tesztelés helyszínéként használtuk. Az ide feltöltött kódok még tesztelés alatt álltak, de a fejlesztési idő alatt mindig a develop verzióját futtattuk az Azure környezetben, ezzel biztosítva, hogy az aktuális fejlesztések folyamatosan elérhetők és tesztelhetők legyenek.
- feature branchek: Az új funkciók fejlesztését különálló, ún. feature branchekben végeztük, amelyek a konkrét feladatokra vagy kódrészletekre koncentráltak. Például kisebb munkaigényű kódok, mint az általános API lekérések fejlesztése külön branchekben történt, míg az összetettebb funkciókat külön feature branch-ekben dolgoztuk ki.

A branch-ek ilyen felosztása lehetővé tette a párhuzamos fejlesztést, és biztosította, hogy a különböző fejlesztési szakaszok kódjai elkülönítve és rendezetten legyenek tárolva. A GitHub pull request funkciójának köszönhetően minden változtatást felül tudtunk vizsgálni, így minden módosítás előtt biztosak lehettünk annak minőségében és kompatibilitásában.



5.4. ábra. A GitHub repository áttekintése

Ezek az eszközök hozzájárultak a projekt sikeres és hatékony fejlesztéséhez, elősegítve a feladatok egyértelműsítését, priorizálását és a csapat tagjai közötti együttműködést.

6. fejezet

Infrastruktúra

6.1. Azure infrastruktúra áttekintése

A Bus4U rendszer a Microsoft Azure felhőalapú platformján került telepítésre. Az Azure szolgáltatások skálázhatóságot és megbízhatóságot biztosítanak a rendszer számára. A telepítéshez az alábbi főbb szolgáltatásokat használtam:

- Azure Virtual Machines (VM): A szerver hosztolása.
- Azure Database for PostgreSQL: Az adatbázis-szolgáltatás biztosítása.

A telepítési infrastruktúrát a 6.1 ábra szemlélteti, amely bemutatja a rendszer főbb komponenseit és azok kapcsolatait.

6.2. Domain konfiguráció

A rendszer az `api.bus4u.online` domainen érhető el, amelyet a Cloudflare platformján konfiguráltam. A domain hozzárendelése során a virtuális gép (VM) IP címét használtam. A következő beállításokat végeztem el:

- DNS beállítások: A Cloudflare-en elvégeztem a domain névhez tartozó A és CNAME rekordok konfigurálását.

- SSL tanúsítványok: A HTTPS alapú kapcsolat biztonságának érdekében a Cloudflare szolgáltatásait alkalmazzuk, amelyek védelmet nyújtanak a felhasználók számára.

6.3. Adatbázis telepítése és konfigurációja

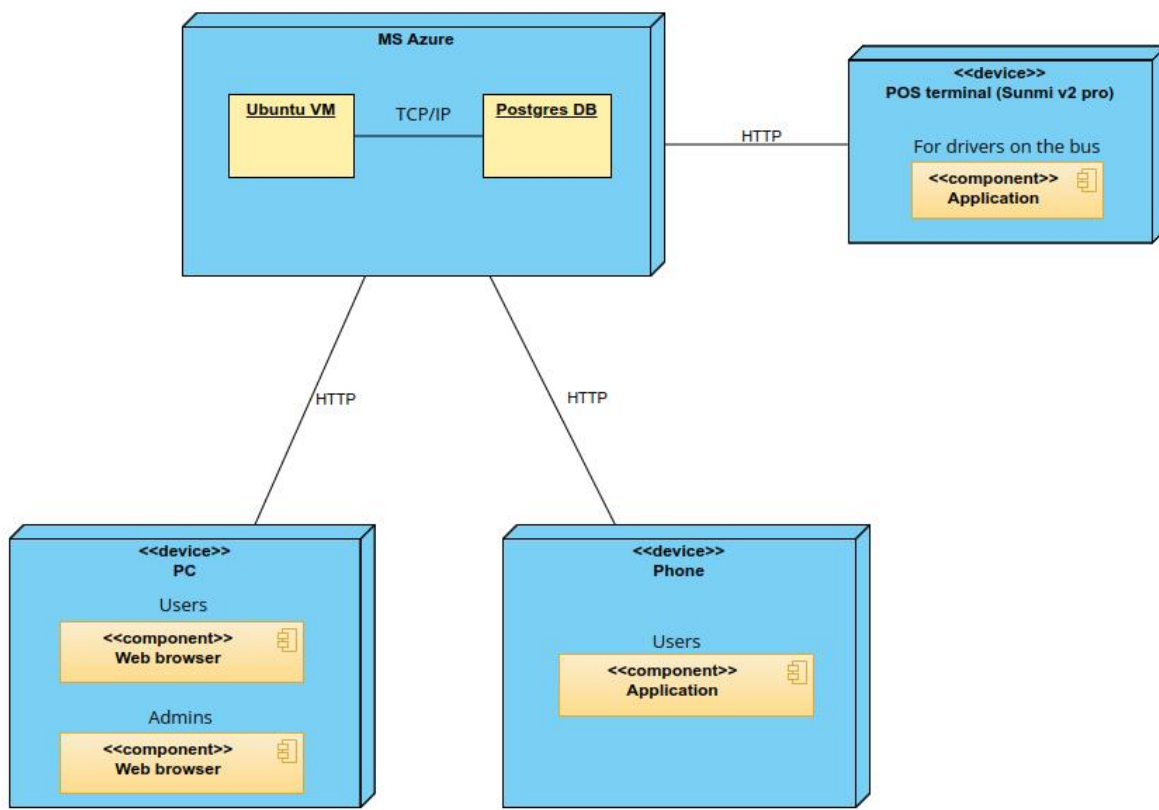
A rendszer adatbázisa egy Azure Database for PostgreSQL szolgáltatáson keresztül került telepítésre, amely lehetővé teszi az adatbázis automatikus méretezését és a folyamatos biztonsági mentések készítését. A következő beállításokat alkalmazzuk:

- Titkosított kapcsolatok: A PostgreSQL adatbázis SSL tanúsítványokat használ a biztonságos kommunikációhoz.
- Backup és recovery: Automatikus biztonsági mentések kerülnek tárolásra az adatvesztés elkerülése érdekében.

6.4. Szerver telepítése

A szerver az Azure Virtual Machines szolgáltatásban került telepítésre, ahol a Ruby on Rails alapú API-t futtatjuk. A telepítés során a következő lépéseket hajtottuk végre:

- Környezeti változók beállítása: A szükséges konfigurációs változók (pl. adatbázis kapcsolatok) a Ruby on Rails beépített credentials funkciójában lettek biztonságosan tárolva és kezelve.
- Verziókezelés: Az asdf verziókezelő használatával a Ruby és egyéb nyelvek verzióit könnyen kezelhetjük, ami megkönnyíti a fejlesztési környezet fenntartását.
- CI/CD pipeline: Az alkalmazás automatikus telepítése és frissítése a GitHub Actions integrációjával valósult meg.



6.1. ábra. Telepítési diagram

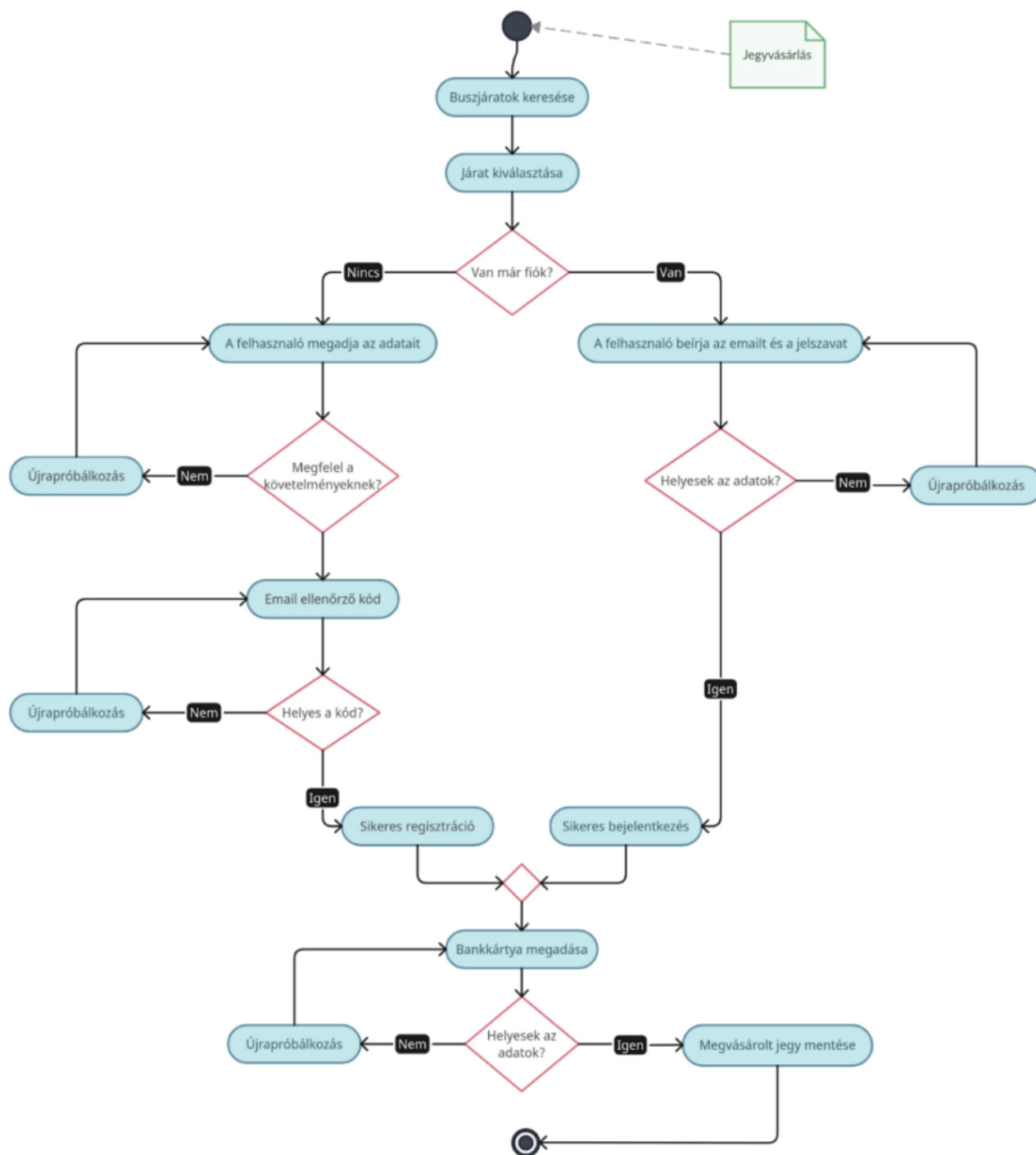
7. fejezet

Az alkalmazás működése

7.1. Felhasználói felület

7.1.1. Főoldal

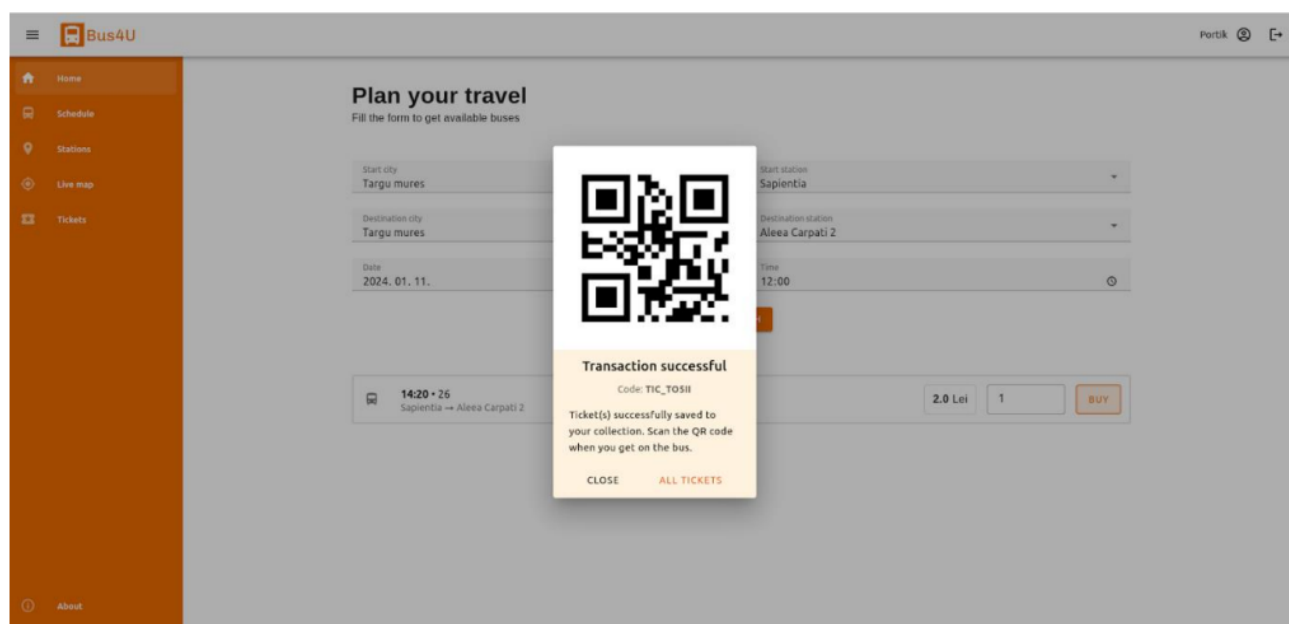
A weboldalra rákeresve a felhasználót a kezdőoldal fogadja, ahol lehetőség van egy utazás megtervezésére. Ki kell választani a kiindulási helyet és megállót, majd a célt, az utazás dátumát és időpontját. Ekkor a rendszer lekéri a szerverről a megadott megállók közötti, az adott napon, a megadott időpont után közlekedő járatokat és azok jegyárát. Minden egyes járatnál van egy vásárlás gomb, amellyel a felhasználó megszerezheti a jegyet vagy jegyeit az adott buszjáratra, amennyiben be van jelentkezve, ellenkező esetben a rendszer átirányítja a bejelentkezéshez.



7.1. ábra. Aktivitás diagram az utazás tervezésének folyamatáról

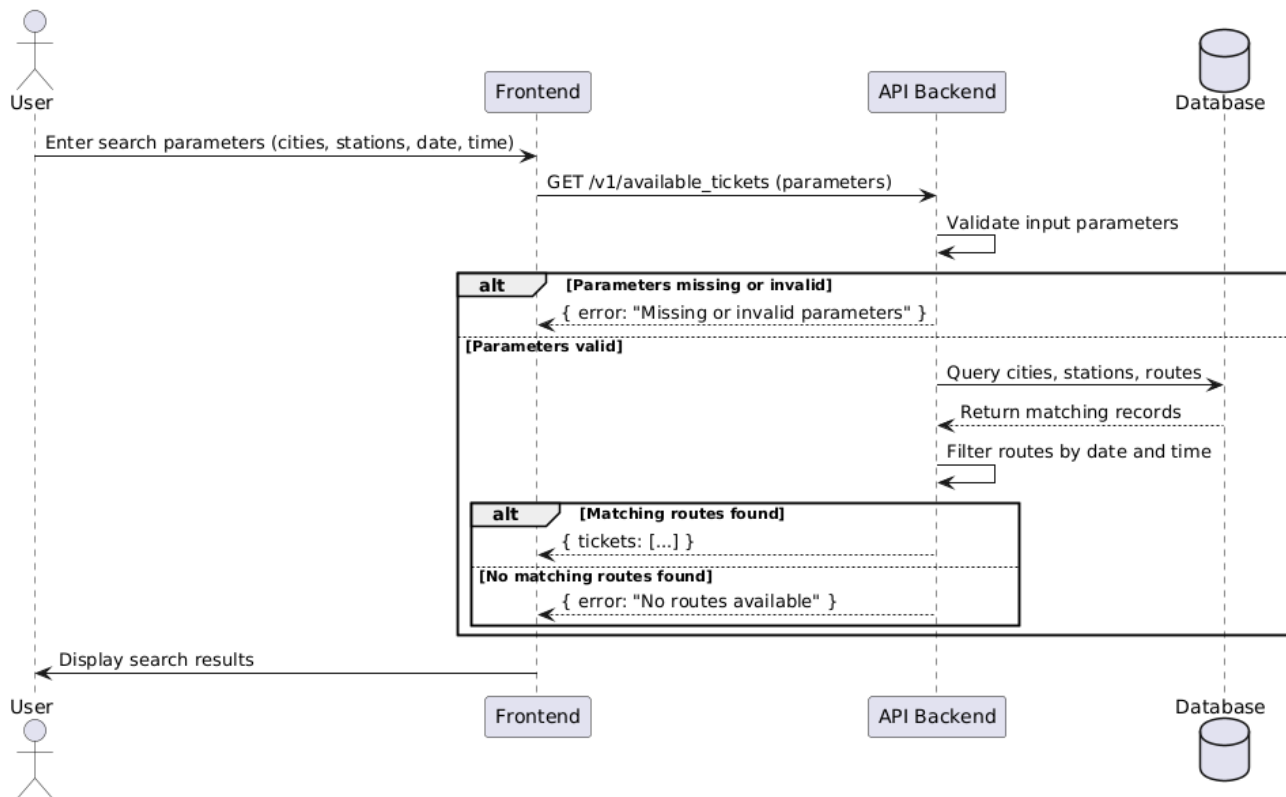
Ahogy a 7.1 ábrán is látható, miután a felhasználó megtalálta a megfelelő buszjáratot és rálépett a jegy(ek) megvásárlására, a rendszer átirányítja őt a bejelentkezéshez, itt ha van már fiókja, beje-

lentkezheth a meglévő adataival, amennyiben nincs, úgy átléphet a regisztrációs oldalra, ahol meg kell adnia a nevét, email címét, és egy új jelszót. A rendszer minden adatot ellenőriz és ha valamelyik hibás (pl. érvénytelen email cím vagy túl rövid jelszó) akkor egy hibaüzenetet ír ki. Ez addig ismétlődik, amíg minden adat megfelelő lesz, aztán automatikusan küldünk egy ellenőrző kódot a megadott e-mailre, amelyet a felhasználó be kell írjon a webes felületen megjelenő szövegmezőbe.



7.2. ábra. Megvásárolt jegy

Ha ez helytelen, ez is addig ismétlődik, amíg minden rendben lesz, majd a rendszer átirányítja a felhasználót a bankkártyás fizetési oldalra, ami jelenleg nincs implementálva, ez a jövőbeli tervek között szerepel. Ezután már csak az következik, hogy a rendszer elmenti a megvásárolt jegy kódját, amelyet a felhasználó a Jegyek oldalon talál meg szöveges és QR-kódos formátumban.

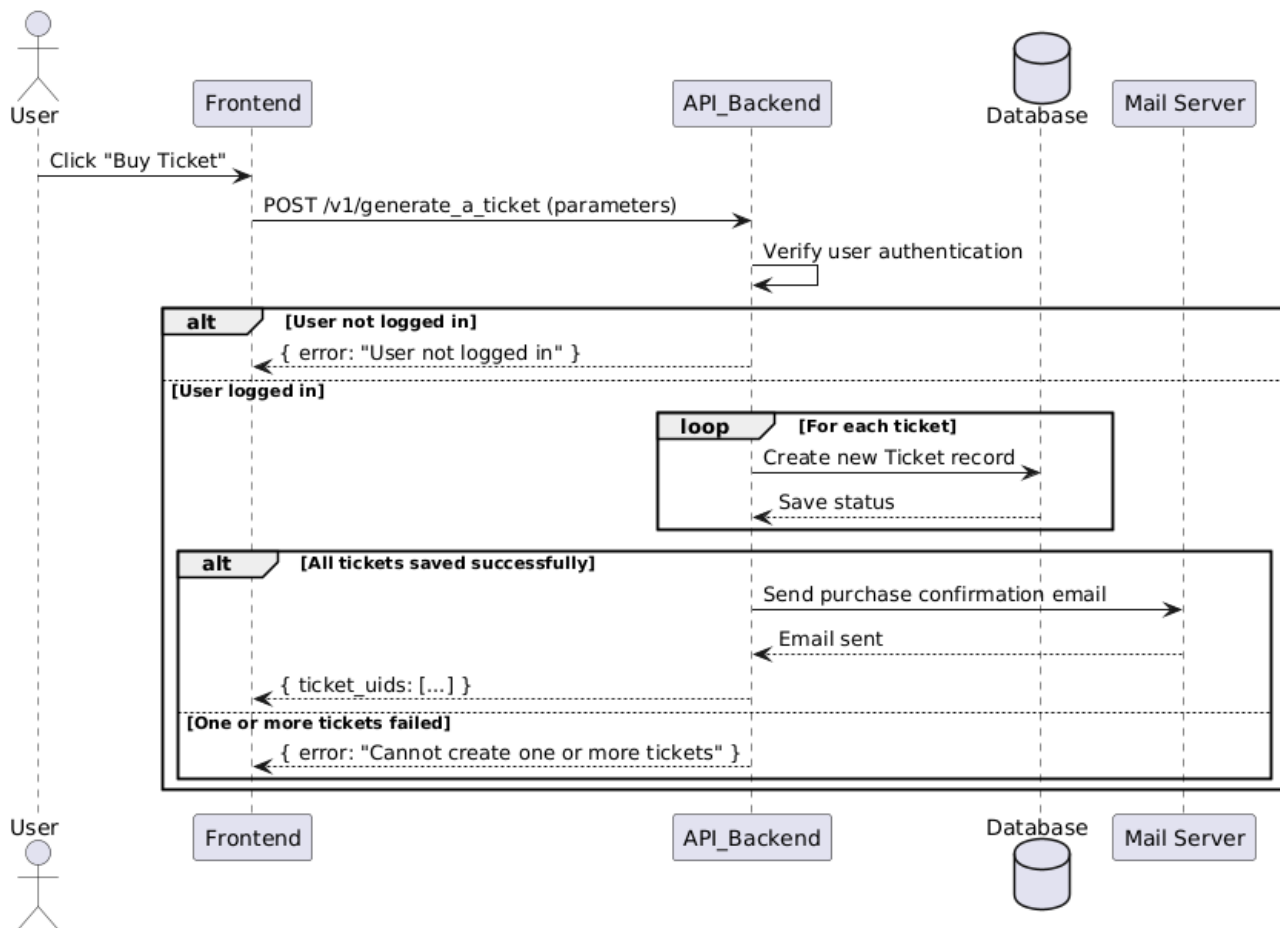


7.3. ábra. Szekvencia diagram az útvonaltervezés folyamatáról

7.1.2. Jegyek

A megvásárolt jegyeket a rendszer elmenti a gyűjteménybe, ami a Jegyek oldalon található, így a jegyek bármikor visszakereshetőek és felhasználhatóak lesznek, kivéve ha lejár az egy hónapos érvényesség.

A jegyen megjelenített adatok között van a járat neve, a kiinduló- és célállomás, az ár, valamint a vásárlás és az érvényesség dátumai. A már elhasznált jegyek megvannak jelölve így azokat nem lehet többször felhasználni. A jegyeket QR-kód formájában is megjelenítjük (7.2 ábra), amelyeket az utasok be tudnak szkennelni.



7.4. ábra. Szekvencia diagram a jegyvásárlás folyamatáról

7.1.3. Menetrend

A következő oldal a menetrend, amely hasonlóan a buszmegállókban kihelyezett táblázatokhoz, megjeleníti a kiválasztott buszjárat indulási időpontjait, az adott megállóból. A mi megoldásunk viszont tartalmaz emellett egy térképet is, ahol meg lehet nézni a járat útvonalát, az érintett megállókat és az útvonalon közlekedő buszok élő pozícióját is.

7.1.4. Megállók és térkép

Ez a két oldal a fent említett térképes funkciókat mutatják meg külön-külön, egyszerűbben. Egyik megjeleníti a térképen az összes megállót, és ezek között szűrést is biztosít város vagy buszjárat sze-

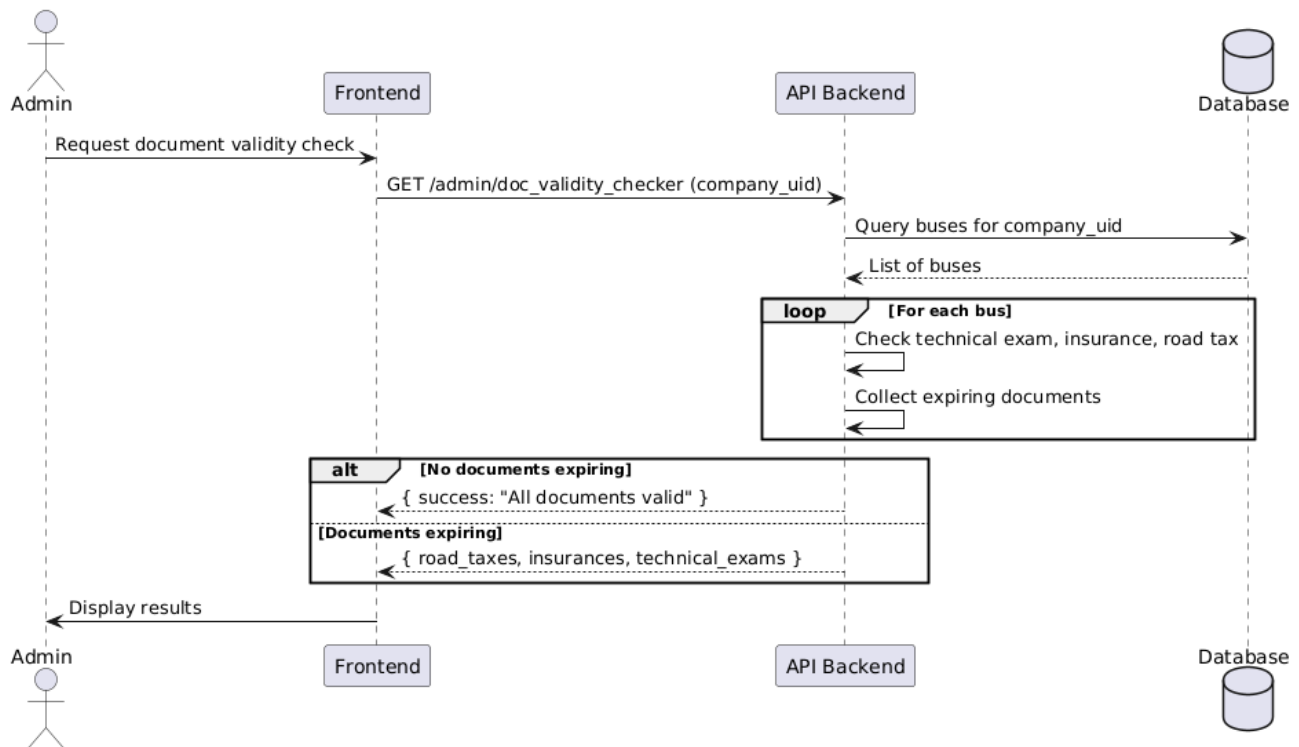
rint. A másik oldalon pedig egy-egy útvonalon közlekedő buszok élő pozícióját lehet követni, miután kiválasztottuk a várost és az útvonalat. Ez a funkció abban segít, hogy ne maradjunk le, ha egy busz siet, illetve ne várakozzunk feleslegesen a megállóban, ha egy busz késik, így időt lehet vele spórolni.

7.2. Adminisztrációs felület

Az admin felület csak bejelentkezéssel elérhető, ehhez egy admin rangú felhasználói fiók szükséges, amely egy adott céghez van hozzárendelve. A cégek adatai el vannak különítve, így minden admin csak a saját cégéhez tartozó adatokat láthatja és módosíthatja.

7.2.1. Kezdőlap

Belépés után egy összegző oldal jelenik meg amelyen látható egy kis statisztika a regisztrált felhasználók számával, a megvásárolt jegyek számával és a bevétellel, mindez havi, évi és mindenkoriban. Itt jelennek meg az esetleges figyelmeztetések és értesítések is, például a buszok lejárt útdíja vagy a közelgő műszaki vizsgálat.



7.5. ábra. Szekvencia diagram a dokumentumok ellenőrzéséről

7.2.2. Megállók

A Megállók oldalon létre lehet hozni új megállókat, megtekinteni meglévőket a térképen vagy törölni ezeket. Új megálló hozzáadásához meg kell adni a megálló nevét, a címet és a koordinátákat. A néven kívül, a többi adatot automatikusan kitölti a rendszer, ha a térképen a megálló helyére kattint a felhasználó, de manuálisan is meg lehet adni.

7.2.3. Útvonalak

Ezen az oldalon lehet felépíteni útvonalakat a létrehozott megállókából és itt lehet megadni az alap jegyárat is, abban az esetben, ha a jegyek ára nem függ a megtett távolságtól és időtől, hanem mindig és mindenhova ugyanannyi, ez esetben a fizetés az útvonalhoz lesz hozzárendelve, nem a különböző megállókhoz.

7.2.4. Menetrend

A következő oldal a Menetrend, ahol a létrehozott útvonalak megállóihoz lehet órarendeket hozzárendelni. Itt meg kell adni az órarendre érvényes napokat, a következő megállóig szükséges jegyárat és az indulási időpontokat. Egy megállóhoz hozzá lehet adni több ilyen órarendet is, ha különböző napokra különböző órarend érvényes. Így el lehet választani a hétvégét a hétköznapoktól, ha mások az indulási időpontok, vagy ha például hétvégén drágább a jegy.

7.2.5. Buszok és alkalmazottak

Az utolsó két oldalon a buszokat és az alkalmazottakat lehet kezelni. Hasonlóan működnek, mindkettőn található egy lista a meglévő elemekkel, ahol bármelyiket lehet módosítani vagy törölni, illetve újat is lehet létrehozni. A buszoknál meg kell adni a márkát, a rendszámot, a férőhelyek számát, a gyártási évet, valamint az útdó, a biztosítás és a műszaki engedély lejárat dátumát. Új alkalmazott hozzáadásánál meg kell adni a nevet, e-mail címet, beosztást, telefonszámot, lakcímet (opcionális), és egy új jelszavat.

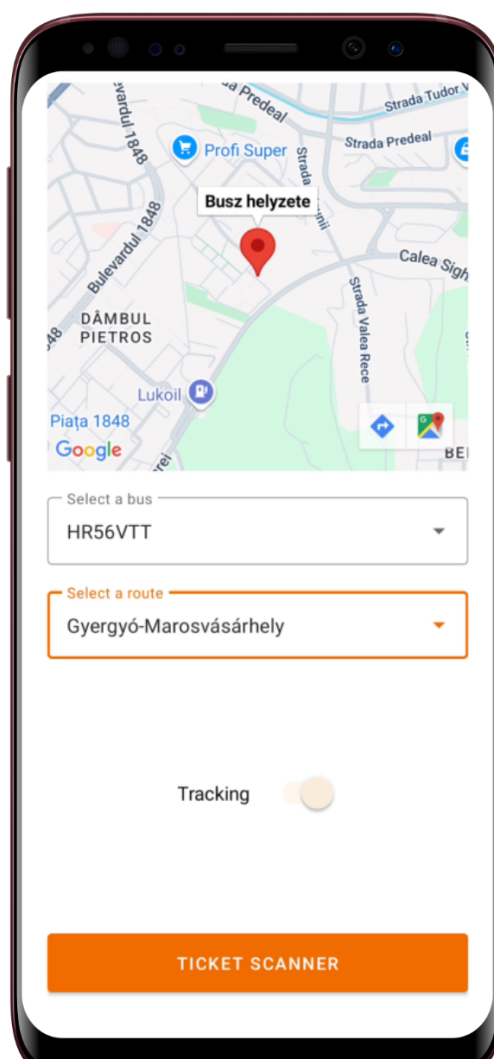
7.3. POS terminál alkalmazása

7.3.1. GPS oldal

Miután a sofőr bejelentkezett, ezen az oldalon lehetőség nyílik kiválasztani az aktuális buszt, amelyiket jelenleg a sofőr vezet, illetve azt az útvonalat, amelyet éppenséggel végrehajt. Található egy kapcsoló, amellyel el lehet indítani az autóbusz lokációjának követését. Ez félpercenként küldi fel az API-ra a busz lokációját, ezáltal lehet a weboldalon és az alkalmazásban élőben követni a busz helyzetét. Illetve ezen opció bekapcsolásakor megjelenik egy térkép is, amelyen élőben láthatjuk a busz pontos helyzetét. Legalul egy Ticket scanner gomb fedezhető fel, amely átirányít a jegyszkennelő oldalra



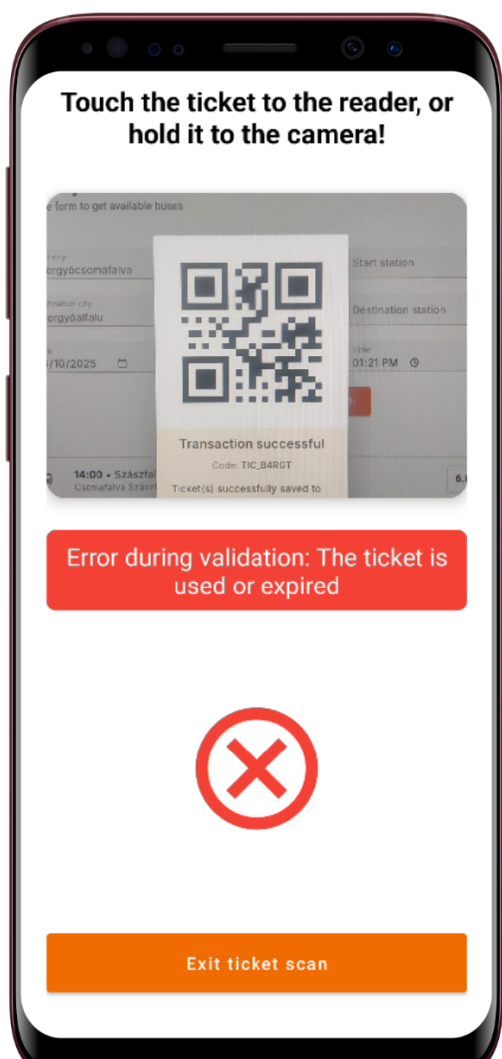
7.6. ábra. Busz kiválasztása és GPS követés



7.7. ábra. Elindított GPS követés

7.3.2. Jegyszkennelő oldal

Ez az oldal a kamerát jeleníti meg. Itt kell beszkenyelni a jegyet. Szkennelés után egy szöveges üzenet jelenik meg 5 másodpercig, három üzenetet hordozhat: Ha sikeresen érvényesítve van a jegy, akkor zöld háttérrel kiírja hogy sikeres érvényesítés. Ha a jegy már érvényesítve volt, vagy érvénytelen a QR kód, ezt egy piros hibaüzenettel jelzi. Illetőleg ha az API valamilyen oknál fogva leáll, akkor ennek a hibájáról is üzenetet kapunk.



7.8. ábra. Sikeres jegy érvényesítés



7.9. ábra. Sikertelen jegy érvényesítés

8. fejezet

A rendszer tesztelése

8.1. Tesztelési környezet

A tesztelést egy helyi fejlesztési környezetben végeztem. A szerver a következő konfigurációval működött:

- Operációs rendszer: Ubuntu 22.04 LTS
- Szerver: Ruby on Rails 7.0.4
- Adatbázis: PostgreSQL 14
- Tesztelő eszközök: RSpec, Postman v10.0, ZAPProxy v2.16

8.2. Funkcionális tesztelés

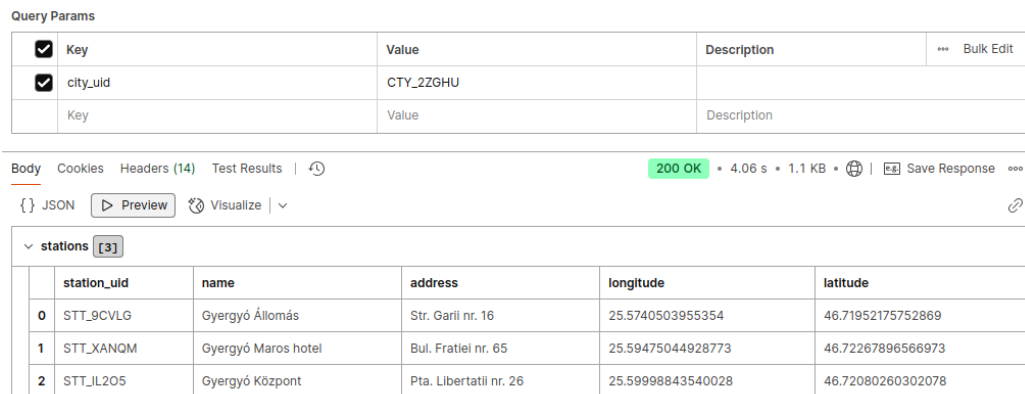
A funkcionális tesztelés során unit teszteket végeztem az RSpec keretrendszer segítségével, valamint az API végpontok működését manuálisan is teszteltem Postman segítségével. Az RSpec lehetőséget biztosít az egyes alkalmazáskomponensek (modellek, kontroller akciók, szolgáltatások) helyességének ellenőrzésére, míg a Postman a különböző API végpontok, azok válaszainak és adatstruktúráinak ellenőrzésére szolgált.

8.2.1. API végpontok tesztelése

A funkcionális tesztelés egyes részeiben a Postman alkalmazást használtam, amely lehetőséget ad HTTP-kérések küldésére, és az API válaszainak kényelmes ellenőrzésére.

Fontosabb végpontok, amelyeket teszteltem:

- GET /v1/get_stations_by_city (8.1 ábra) – Ez az API végpont lehetővé teszi, hogy egy adott városhoz tartozó állomásokat kérjünk le. Ezen végpont segítségével a felhasználók gyorsan megtalálhatják azokat az állomásokat, amelyek egy meghatározott városhoz tartoznak. A kérés paraméterei tartalmazzák a város nevét, és a válasz egy listát ad vissza az állomásokról.



Query Params

☒ Key	Value	Description	*** Bulk Edit
☒ city_uid	CTY_2ZGHU		
Key	Value	Description	

Body Cookies Headers (14) Test Results | 200 OK • 4.06 s • 1.1 KB | Save Response

{ } JSON Preview Visualize

stations [3]

	station_uid	name	address	longitude	latitude
0	STT_9CVLG	Gyergyó Állomás	Str. Garil nr. 16	25.5740503955354	46.71952175752869
1	STT_XANQM	Gyergyó Maros hotel	Bul. Fratiei nr. 65	25.59475044928773	46.72267896566973
2	STT_IL205	Gyergyó Központ	Pta. Libertatii nr. 26	25.59998843540028	46.72080260302078

8.1. ábra. A GET /v1/get_stations_by_city végpont kérésének paraméterei és válasza

- GET /v1/get_available_tickets (8.2 ábra) – Ez a végpont lehetővé teszi az elérhető jegyek lekérdezését egy adott kiindulási és célállomás között. A kérés paraméterei a kiindulási és célállomás, valamint egy opcionális időpont. A megállók is opcionálisak, a csak a város megadása esetén minden megállóhoz láthatjuk a jegyeket. A válasz tartalmazza az elérhető jegyek listáját, és ezek árai, valamint egyéb információkat.
- GET /v1/get_bus_locations_by_route (8.3 ábra) – Ezen API végpont segítségével lekérhetjük a buszok aktuális helyzetét egy adott járatra vonatkozóan. A kérés tartalmazza a járat azonosítóját, és a válasz egy koordinátákkal mutatja a buszok aktuális pozícióit.

Query Params

☐	Key	Value	Description	...	Bulk Edit
☒	start_city_uid	CTY_2ZGHU			
☐	start_station_uid	STT_D2Y7W			
☒	destination_city_uid	CTY_CCTK2			
☐	destination_station_uid	STT_O172E			
☒	date	2023-12-21			
☒	time	16:00			
	Key	Value	Description		

Body

Cookies

Headers (14)

Test Results

200 OK

6.36 s

1.84 KB

Save Response

{ }

JSON

Preview

Visualize

	from_station	from_station_uid	to_station	to_station_uid	company_name	company_uid	route_name	route_uid	ticket_price	departure_time
0	Gyergyó Állomás	STT_9CVLG	Dedeman	STT_W848K	VANDOR TRANS TOURS SRL	CPY_MNPCM	Gyergyó-Marosvásárhely	ROU_I0BF3	38.0	1743098700
1	Gyergyó Állomás	STT_9CVLG	Piata Trandafirilor	STT_RNUUP	VANDOR TRANS TOURS SRL	CPY_MNPCM	Gyergyó-Marosvásárhely	ROU_I0BF3	39.0	1743098700
2	Gyergyó Központ	STT_IL2O5	Dedeman	STT_W848K	VANDOR TRANS TOURS SRL	CPY_MNPCM	Gyergyó-Marosvásárhely	ROU_I0BF3	40.0	1743098400
3	Gyergyó Központ	STT_IL2O5	Piata Trandafirilor	STT_RNUUP	VANDOR TRANS TOURS SRL	CPY_MNPCM	Gyergyó-Marosvásárhely	ROU_I0BF3	41.0	1743098400

8.2. ábra. A GET /v1/get_available_tickets végpont kérésének paraméterei és válasza

Query Params									
☒	Key	Value	Description	Bulk Edit					
☒	route_uid	ROU_I0BF3							
	Key	Value	Description						

Body Cookies Headers (14) Test Results 200 OK • 591 ms • 1.08 KB Save Response									
{ } JSON Preview Visualize									
	bus_uid	company_uid	license_plate	brand	manufacturing_year	capacity	road_tax	insurance	technical_exam
0	BUS_GDP1W	CPY_MNPCM	HR12VTT	Otokar Sultan	2011	24	2024-06-29T00:00:00.000+03:00	2024-01-25T00:00:00.000+02:00	2024-06-11T00:00:00.000+03

8.3. ábra. A GET /v1/get_bus_locations_by_route végpont kérésének paraméterei és válasza

- GET /v1/tickets_of_user (8.4 ábra) – Az aktuális felhasználó jegyeinek lekérésére szolgáló végpont. A kérés nem tartalmaz paramétereket, mivel az aktuális felhasználó jegyeit kérdezi le. A válaszban szereplő jegyek információi tartalmazzák a jegyek típusát, érvényességüket és egyéb fontos adatokat.
- POST /v1/admin/add_station_to_route (8.5 ábra) – Ez az adminisztrátorok számára elérhető végpont lehetővé teszi egy állomás hozzárendelését egy adott útvonalhoz. A kérés tartalmazza az útvonal és az állomás azonosítóját, és a válasz megerősíti a sikeres műveletet.

Body	Cookies (1)	Headers (21)	Test Results		🔍	200 OK	• 1.56 s • 2.22 KB • 🌐 📄 Save Response	...
{ }	JSON	▶ Preview	🔗 Visualize		▼			🔗
Root / 0								
ticket_uid	TIC_4DCMG							
date_of_purchase	2025-03-26T18:57:43.414+02:00							
expiration_date	2025-04-26T19:57:43.414+03:00							
from_station_uid	STT_9CVLG							
to_station_uid	STT_MBGDP							
from_station_name	Gyergyó Állomás							
to_station_name	Csomafalu Szászfalu							
is_valid	true							
route_uid	ROU_28JLW							
route_name	Szászfalu-Gyergyó							
ticket_price	1100.0							

8.4. ábra. A GET /v1/tickets_of_user végpont kérésének paraméterei és válasza

Query Params			
☒	Key	Value	Description
☒	license_plate	HR12VTT	
☒	route_uid	ROU_J0BF3	
	Key	Value	Description
Body	Cookies (1)	Headers (21)	Test Results
			200 OK • 1.12 s • 1.68 KB • 🌐 📄 Save Response
{ }	JSON	▶ Preview	🔗 Visualize
success	Current route saved successfully!		

8.5. ábra. A POST /v1/admin/add_station_to_route végpont kérésének paraméterei és válasza

A Postman segítségével végzett tesztelés eredményeként az összes API végpont sikeresen válaszolt a kérésekre, és az adatstruktúrák megfeleltek az elvárt formátumnak.

8.2.2. Unit tesztek

Az RSpec segítségével végzett unit tesztelés során az alkalmazás különböző komponenseit, mint például a modelleket, illetve kontrollereket teszteltem. Az alábbiakban bemutatok egy példát az RSpec tesztre, amely a GET /v1/get_stations_by_city végpontot teszteli:

```
describe 'GET /v1/get_stations_by_city' do
  let(:city_uid) { 'CTY_N11XH' }
  let(:valid_url) { "/v1/get_stations_by_city?city_uid=#{city_uid}" }
```

```

it 'returns a list of stations' do
  get valid_url
  expect(response).to have_http_status(:ok)
  expect(response.body).to include('stations')
end

it 'returns valid station data' do
  get valid_url
  expect(response.body).to include('station_uid')
  expect(response.body).to include('name')
  expect(response.body).to include('address')
end
end

```

Ez a teszt ellenőrzi, hogy a GET /v1/get_stations_by_city végpont megfelelően válaszol-e a kért városra vonatkozóan, és hogy az adatok (pl. station_uid, name, address) jelen vannak-e a válaszban.

8.3. Biztonsági tesztelés

A biztonsági teszteléshez az OWASP ZAP (Zed Attack Proxy) eszközt használtam, amely egy nyílt forráskódú, automatizált webalkalmazás biztonsági tesztelő eszköz. A ZAP segítségével alaposan átvizsgáltam az API-t és annak különböző végpontjait a fent említett sebezhetőségek szempontjából.

8.3.1. SQL Injection

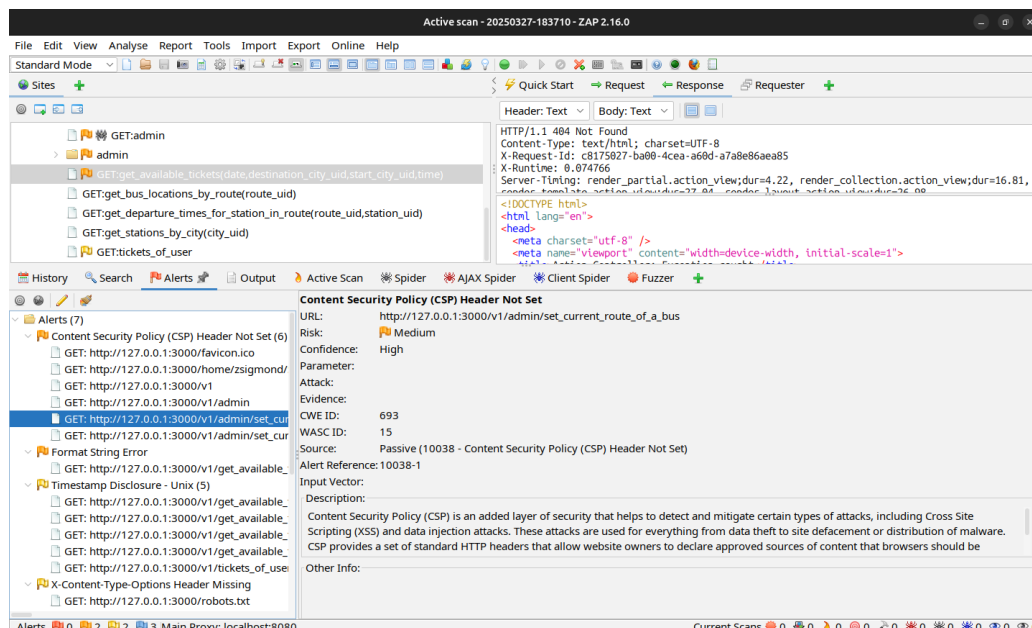
Az SQL injection sebezhetőségek elleni védelem érdekében minden felhasználói bemenetet megfelelően validáltam és szűrtem. Az alkalmazásban használt ActiveRecord ORM automatikusan védi az alkalmazást az SQL injection támadásokkal szemben, mivel előkészített SQL lekérdezéseket alkal-

maz, amelyek biztosítják, hogy a felhasználói adatok ne manipulálják az SQL parancsokat. A ZAP automatizált szkennelő funkcióját használtam, hogy szimuláljam a különböző SQL injection támadásokat, például az OR 1=1 típusú támadásokat. A szkennер jelzései alapján megerősítettem, hogy minden SQL lekérdezés biztonságosan van előkészítve, és semmilyen dinamikusan generált SQL parancs nem futtatható a bemeneti értékek alapján.

8.3.2. XSS (Cross-Site Scripting) Tesztelés

A cross-site scripting (XSS) támadások megelőzése érdekében minden felhasználói bemenetet úgy kezeltem, hogy ne engedje a HTML vagy JavaScript kódok végrehajtását. A ZAP XSS szkennер funkciója során észleltem, hogy a rendszer megfelelően kódolja az összes HTML entitást, és semmilyen felhasználói bemenetet nem engedett közvetlenül a HTML válaszba. A ZAP által futtatott támadások eredményeként biztosítottam, hogy az API nem engedélyezi a szkriptek vagy más veszélyes elemek beágyazását az oldalakba.

8.3.3. Felfedezett sebezhetőségek



8.6. ábra. A ZAP által felfedezett sebezhetőségek listája

Format String Error

A Format String Error akkor fordul elő, amikor a bemeneti adatokat az alkalmazás parancsként értékeli ki, különösen formátumú stringek esetén (pl. %s, %d, stb.). Az ilyen típusú hibák lehetőséget adnak támadók számára, hogy manipulálják az alkalmazás működését, például memóriát írjanak felül vagy kódfuttatási hibákat okozzanak.

A következő URL-t teszteltem, amely format string karaktereket tartalmazott:

```
http://127.0.0.1:3000/v1/get_available_tickets
```

A bemeneti értékekben szereplő formátum karakterek (pl. %s, %n) potenciálisan kihasználhatók arra, hogy a támadó manipulálja az alkalmazás működését vagy hibás adatokat szerezzen. Az alkalmazás minden bemenetet megfelelően validál és a dinamikus formázást nem engedélyezi, biztosítva a védekezést a format string hibák ellen.

Timestamp Disclosure

A Timestamp Disclosure azt jelenti, hogy az alkalmazás vagy a webkiszolgáló olyan érzékeny információkat, mint a rendszer időbélyege (Unix timestamp), kiszivároztat a válaszban. Ez az információ segíthet a támadóknak abban, hogy pontosabb támadásokat hajtsanak végre, például brute-force támadásokkal.

Az alábbi URL-ben észleltem, hogy az alkalmazás Unix timestampet szolgáltatott:

```
http://127.0.0.1:3000/v1/get_available_tickets
```

Ez az információ potenciálisan érzékeny, és elárulhatja a rendszer belső működését, lehetőséget adva támadóknak arra, hogy pontosabb támadásokat hajtsanak végre, mint például brute-force támadások. Az alkalmazásban ezt az információt nem szabadna kiszivároztatni, és biztosítani kell, hogy minden érzékeny adat titkosítva vagy megfelelően elrejtve legyen a válaszokban.

Anti-MIME-Sniffing és X-Content-Type-Options

A X-Content-Type-Options fejléc beállítása elengedhetetlen a MIME típusok szaglásának megakadályozására. A MIME szaglás lehetővé teszi a böngészők számára, hogy egy nem várt tartalom típust próbáljanak kitalálni, ha nem találják meg a Content-Type fejlécet, amely segíthet sérülékenységeket

kihasználó támadásokban, mint például a XSS támadások.

A következő konfigurációt alkalmaztam a webalkalmazásban a MIME típusok pontos meghatározására:

```
X-Content-Type-Options: nosniff
```

Ez biztosítja, hogy a böngészők ne végezzenek MIME-szaglást, és ne próbáljanak kitalálni más tartalom típust, ami növeli a webalkalmazás biztonságát.

robots.txt

A `robots.txt` fájlban is megtettem a szükséges intézkedéseket a biztonság érdekében, biztosítva, hogy a webalkalmazás ne adjon ki érzékeny információkat a keresőrobotoknak. A következő beállításokat alkalmaztam:

```
User-agent: * Disallow: /admin/ Disallow: /config/
```

Ez a konfiguráció biztosítja, hogy a keresőrobotok ne indexeljék a bizalmas oldalak és fájlok tartalmát, amelyek a felhasználók adatainak védelmét szolgálják.

9. fejezet

Összefoglalás

Ez a fejezet összefoglalja a Bus4U rendszer API-jának tervezését, megvalósítását és tesztelését, valamint bemutatja a legfontosabb megállapításokat és javaslatokat a jövőbeli fejlesztésekhez.

A dolgozat célja egy hatékony busz menedzselő rendszer API létrehozása volt, amely lehetővé teszi a felhasználók számára a buszjáratok, jegyek és bérletek kezelését, valamint a rendszeres frissítéseket és karbantartásokat. A projekt során a következő lépéseket tettük:

1. Követelmények és tervezés: A rendszer funkcionális és nem funkcionális követelményeinek részletes elemzése és a legfontosabb funkciók prioritásának meghatározása. A követelmények alapján a rendszer architektúráját és az API felépítését terveztük meg.

2. API megvalósítása: Az API endpointok megvalósítása során külön figyelmet fordítottunk a RESTful architektúra elveinek betartására, biztosítva ezzel a könnyű használhatóságot és a skálázhatóságot. A végpontok különböző erőforrásokat (buszok, járatok, jegyek, bérletek) kezelnek, lehetővé téve a felhasználók számára, hogy egyszerűen hozzáférjenek az információkhoz és végrehajtsanak műveleteket.

3. Tesztelés: A rendszer tesztelését Postman segítségével végeztük, ahol különböző kéréseket küldtünk az API végpontokhoz, hogy ellenőrizzük azok működését, a válaszok helyességét és a teljesítményt. A tesztelési folyamat során sikerült azonosítani és javítani a hibákat, így a rendszer stabil és megbízható lett.

4. Deployment: A rendszer telepítése az Azure felhőszolgáltatásra történt, ahol az API az `api.bus4u.online`

domainen érhető el. A PostgreSQL adatbázis és a virtuális gép (VM) külön szolgáltatásként működik, biztosítva a skálázhatóságot és a megbízhatóságot.

5. Jövőbeli fejlesztések: A jövőbeli fejlesztések között szerepel a rendszer funkcionalitásának bővítése, például új szolgáltatások bevezetése, a felhasználói élmény javítása érdekében. További cél a rendszer biztonságának növelése, például a hitelesítési és jogosultsági mechanizmusok fejlesztésével.

Összességében a Bus4U API megvalósítása sikeres volt, és a dolgozat során végrehajtott lépések hozzájárultak a hatékony és megbízható busz menedzselő rendszer kialakításához. A projekt során szerzett tapasztalatok és tanulságok értékes alapot jelentenek a jövőbeli fejlesztésekhez és a hasonló rendszerek tervezéséhez.

Irodalomjegyzék

- [1] P. Szabolcs, „Bus4u - ui,” bachelor’s thesis, Sapientia Hungarian University of Transylvania, 2025.
- [2] D. M. Levinson and K. J. Krizek, *The End of Traffic and the Future of Transport*. Network Design Lab, 2016.
- [3] A. Ceder, *Public Transit Planning and Operation*. Elsevier, 2016.
- [4] W. Zhou, L. Li, M. Luo, and W. Chou, „Rest api design patterns for sdn northbound api,” in *2014 28th international conference on advanced information networking and applications workshops*, pp. 358–365, IEEE, 2014.
- [5] O. Al-Debagy and P. Martinek, „A comparative review of microservices and monolithic architectures,” in *2018 IEEE 18th International Symposium on Computational Intelligence and Informatics (CINTI)*, pp. 000149–000154, IEEE, 2018.
- [6] M. Hartl, *Ruby on Rails Tutorial: Learn Web Development with Rails*. Addison-Wesley Professional, 2016.
- [7] O. Fernandez, *The Rails Way*. Addison-Wesley Professional, 2017.
- [8] K. Douglas and S. Douglas, *PostgreSQL: Up and Running*. O’Reilly Media, 2015.
- [9] J. Murrell, *PostgreSQL 12 High Availability Cookbook*. Packt Publishing, 2019.
- [10] B. Wilder, *Cloud architecture patterns: using Microsoft Azure*. O’Reilly Media, Inc., 2012.

- [11] M. Collier and R. Shahan, *Microsoft Azure Essentials-Fundamentals of Azure*. Microsoft Press, 2015.

A függelék

Függelék

függelék